G901:A-0005243446

다수의 TCP/UDP Flow 환경상에서 FRED 알고리즘의 공정성 실험적 평가 및 분석

A Simulation Study for FRED in Fairness Provision with Many TCP/UDP Flows

2004년

서강대학교 정보통신대학원

정보처리 전공

김 종 인

다수의 TCP/UDP Flow 환경상에서 FRED 알고리즘의 공정성 실험적 평가 및 분석

A Simulation Study for FRED in Fairness Provision with Many TCP/UDP Flows

지도교수 장 형 수

이 논문을 공학석사 학위논문으로 제출함

2005년 6월 13일

서강대학교 정보통신대학원

정보처리 전공

김 종 인

감사의 글

대학원 생활 동안 감사의 말을 하지 못한 분들께 적은 지면을 빌어 감사의 마음을 전하고자 합니다. 이 논문을 무사히 마칠 수 있도록 부족한 저를 지도해 주신 장형수 교수님과 논문 심사를 해주신 최명환 교수님, 임인성 교수님께 진심으로 감사를 드립니다. 또한 지난 대학원 생활 동안 학문의 깊이를 일깨워 주신 황선영 교수님, 서정연 교수님, 낭종호 교수님, 박성용 교수님, 장주욱 교수님, 김경환 교수님께도 깊은 감사를 드립니다. 그리고 바쁘신 와중에도 많은 조언과 도움을 주신 이인규 선배님께도 감사의 마음을 전합니다.

지난 원우회 임원 활동기간 동안 잊지 못할 추억을 만들어 주신 희대형, 장중형, 장식형, 은희누나, 규상씨, 정은씨에게도 감사의 마음을 전합니다. 입학해서 지금까지 함께 했던 28기 동기 여러분들께도 감사의 말씀을 드립니다. 직장 생활과 병행 했던 학업을 포기 하지 않고 무사히 끝맺을 수 있도록 도와 주신 박상복 이사님, 김영환 과장님께도 감사를 드립니다. 그리고 지금 까지를 저를 도와 주신 모든 분들께 머리 숙여 감사 드립니다. 마지막으로 언제나 저에게 힘이 되어주신 부모님과 동생, 그리고 나의 사랑스러운 아내에게 깊은 감사의 마음을 표합니다.

2005년 6월

김 종 인 드림.

차 례

제 1 장 서 론 1
제 1 절 연구 배경 1
제 2 절 연구 목적 및 제안 기법 2

제 2 장 관련 연구 5
제 1 절 전송 프로토콜 5
(1) TCP (Transmission Control Protocol) 5
(2) UDP (User Datagram Protocol) 6
제 2 절 종단간 혼잡 제어 8
(1) TCP 혼잡 제어 기법 8
(2) TCP 버전에 따른 혼잡 제어 기법 14
(3) TFRC (TCP-Friendly Rate Control) 18

제 3 장 FRED 알고리즘 분석 21
제 1 절 기존 라우터 버퍼 알고리즘 21
(1) DT (Drop Tail) 21
(2) ERD (Early Random Drop) 21
(3) RED (Random Early Detection) 22
(4) ARED (Adaptive Random Early Detection) 27
제 2 절 FRED (Flow Random Early Detection) 28

(1) 기존 알고리즘의 문제점 28
(2) FRED 알고리즘 분석 32
(3) FRED 알고리즘의 문제점 36

제 4 장 실험 결과 및 분석 49
제 1 절 실험 환경 49
제 2 절 실험 결과 55
제 3 절 실험 분석 65

제 5 장 결 론 68
[참고 문헌] 71

표 차례

표 3-1. 평균 큐 크기 계산식 (RED) 24
표 3-2. 평균 버퍼 크기 계산에 사용되는 파라메터 정의 (RED) 24
표 3-3. 패킷 폐기 확률에 사용되는 파라메터 (RED) 25
표 3-4. 패킷 폐기 확률값 계산식 (RED) 25
표 3-5. 그 밖의 시간 관련 파라메터 (RED) 27
표 3-6. 평균 대역폭 점유율 (RED) 30
표 3-7. 평균 대역폭 점유율 (FRED) 30
표 3-8. 분산 및 표준편차 (RED) 31
표 3-9. 분산 및 표준편차 (FRED) 31
표 3-10. 매핑 함수 (FRED) 33
표 3-11. 상수 파라메터 (FRED) 34
표 3-12. 전역 파라메터 (FRED) 34
표 3-13. 플로우 당 파라메터 (RED) 34
표 3-14. many 옵션 미사용 경우와 many 옵션 사용시 비교 40
표 3-15. 다수의 TCP/UDP 기반의 공정성 확인을 위한 시나리오 43
표 3-16. 평균 대역폭 점유율, 분산 및 평균 큐 점유 크기, 분산 46
표 4-1. 시뮬레이션을 위한 시나리오 구성 50
표 4-2. 시뮬레이션을 위한 FRED, Many-FRED 파라메터 값 51
표 4-3. TCP 버전 별 32개 플로우 연결에 의한 비교 55
표 4-4. TCP 버전 별 64개 플로우 연결에 의한 비교 55

표 4-5. TCP 버전 별 128개 플로우 연결에 의한 비교 55
표 4-6. Many-FRED 적용 시 연결 개수에 의한 비교 62

그림 차례

그림 2-1. TCP 세그먼트 구조 6
그림 2-2. UDP 세그먼트 구조 7
그림 2-3. AIMD 알고리즘 동작 예 9
그림 2-4. TCP 에서 Slow Start 동작 11
그림 2-5. RTT에 따른 혼잡 윈도우 변화 12
그림 2-6. TCP/Tahoe와 TCP/Reno 환경에서의 혼잡 윈도우 변화 16
그림 2-7. TFRC에서 사용되는 방정식 19
그림 3-1. 전역 동기화 (Global Synchronization) 22
그림 3-2. RED 알고리즘의 패킷 폐기 확률 변화 25
그림 3-3. RED 알고리즘 의사 코드 26
그림 3-4. TCP/UDP 플로우 연결을 위한 토폴로지 29
그림 3-5. TCP/UDP 플로우에 대한 대역폭 비교 (RED) 29
그림 3-6. TCP/UDP 플로우에 대한 대역폭 비교 (FRED) 30
그림 3-7. RED와 FRED의 큐 점유 상태 비교 31
그림 3-8. FRED 알고리즘 의사 코드 35
그림 3-9. 16개의 TCP 플로우 기반의 토폴로지 38
그림 3-10. 8개의 TCP 연결들의 대역폭 비교 (TCP #16/FRED) 38
그림 3-11. 8개의 TCP 연결들의 대역폭 비교 (TCP #16/many) 39
그림 3-12. many 옵션 미사용 시와 many 옵션 사용 시 혼잡 윈도우 40
그림 3-13. “two packet buffer” 의사 코드 41

그림 3-14. TCP의 Exponential Backoff 42
그림 3-15. TCP 플로우 기반의 토폴로지 (FRED/many) 44
그림 3-16. TCP/UDP 플로우 기반의 토폴로지 (FRED/many)..... 44
그림 3-17. TCP 플로우 기반의 첫번째 TCP 플로우 대역폭 점유 45
그림 3-18. TCP/UDP 플로우 기반의 첫번째 TCP 플로우 대역폭 점유 . 45
그림 3-19. 32개의 TCP 플로우 연결들의 대역폭 점유 (many)..... 47
그림 3-20. 32개의 TCP/UDP 플로우 연결들의 대역폭 점유 (many) 47
그림 4-1. 다수의 TCP/UDP 플로우 기반의 토폴로지..... 50
그림 4-2. Many-FRED를 위한 “specific random drop” 의사코드 54
그림 4-3. 다수 플로우 기반의 TCP와 UDP플로우 사이의 대역폭 분산 . 56
그림 4-4. TCP 플로우의 대역폭 변화 (Tahoe/Flows #32)..... 57
그림 4-5. TCP 플로우의 대역폭 변화 (Tahoe/Flows #64)..... 58
그림 4-6. TCP 플로우의 대역폭 변화 (Tahoe/Flows #128)..... 58
그림 4-7. TCP 플로우의 대역폭 변화 (Newreno/Flows #32) 59
그림 4-8. TCP 플로우의 대역폭 변화 (Newreno/Flows #64) 59
그림 4-9. TCP 플로우의 대역폭 변화 (Newreno/Flows #128) 60
그림 4-10. TCP 플로우의 대역폭 변화 (Vegas/Flows #32)..... 60
그림 4-11. TCP 플로우의 대역폭 변화 (Vegas/Flows #64)..... 61
그림 4-12. TCP 플로우의 대역폭 변화 (Vegas/Flows #128)..... 61
그림 4-13. FRED와 Many-FRED의 TCP/UDP 플로우간 분산 비교 63
그림 4-14. TCP 플로우의 대역폭 변화 (Many-FRED/Flows #32)..... 64
그림 4-15. TCP 플로우의 대역폭 변화 (Many-FRED/Flows #64)..... 64

그림 4-16. TCP 플로우의 대역폭 변화 (Many-FRED/Flows #128)..... 65

Abstract

This thesis studies by simulation the performance comparison of the two router buffer management algorithms, RED and FRED, in fairness provisioning for many TCP/UDP flows.
We evaluate the effectiveness of FRED and show the fairness problems caused by FRED. To resolve the fairness problems, we extend FRED into an algorithm called “Many-FRED” and compare the performance of the extended FRED with RED and the original FRED. FRED includes “many” option to leverage the fairness problems by making it possible to use “two packet buffer” algorithm within the many flow connections. Many-FRED differentiates TCP traffic which controls the stream with the packet header information before the FRED router enters the “two packet buffer” stage by many flow connections from UDP traffic which has no flow control. TCP traffic enters into “two packet buffer” stage like FRED. However, for UDP traffic, if the packet left on the assigned flow buffer is more than two and the size of the average buffer is between minimum and maximum threshold, it is dropped probabilistically by a certain equation. If the average buffer size is over the maximum threshold, “specific random drop” is proposed. We evaluate Many-FRED with a topology based on many TCP/UDP flow for the different versions of TCP and existing FRED.

초 록

본 논문에서는 다수의 TCP/UDP Flow 환경에서 라우터 버퍼 관리 알고리즘인 RED와 FRED에 대해 공정성 실험적 평가 및 분석을 한다.
FRED의 성능을 평가하고 FRED에서 야기 되는 공정성 문제를 제시 한다. 공정성 문제를 해결하기 위해 “Many-FRED”이라 불리는 알고리즘을 통해 FRED를 확장하였고, RED, FRED와 더불어 확장된 FRED의 성능을 비교하였다. FRED는 다수의 플로우 연결에 대해 공정성 문제를 해결하기 위해서 “two packet buffer” 알고리즘을 사용 할 수 있도록 “many” 옵션을 포함하고 있다. Many-FRED는 FRED 라우터가 다수의 플로우 연결에 의한 “two packet buffer” 단계로 진입 전에, 패킷 헤더 정보를 통해 흐름 제어를 하는 TCP 트래픽과 하지 않는 UDP 트래픽을 구별 하여, TCP 트래픽의 경우는 이전 FRED와 동일하게 “two packet buffer” 단계로 들어가게 하고, UDP 트래픽인 경우에는 해당 플로우 버퍼에 남은 패킷이 2개 이상이고 현재 평균 버퍼 크기가 최소 임계치(Minimum Threshold) 에서 최대 임계치(Maximum Threshold) 사이에 해당하면 RED에서 제공하는 확률 계산식에 따라 폐기하고, 현재 평균 버퍼 크기가 최대 임계치를 초과하는 경우에는 UDP 트래픽을 무조건 폐기하는 방법을 사용하는 “specific random drop”을 제안한다. 마지막으로 다수의 TCP/UDP 플로우 기반의 토폴로지를 구성하여 기존 FRED 기반에서 TCP 버전에 따라 시뮬레이션 후 “specific random drop”에 의해 확장된 Many-FRED를 적용하여 그 결과를 토대로 실험적으로 분석하고 평가한다.

제 1 장 서 론

제 1 절 연구 배경

오늘날 인터넷 사용자들의 증가와 실시간 멀티미디어 데이터의 요구가 급격하게 늘어나고 있다. 이와 같은 이유로 인터넷 상에서 실시간 오디오, 비디오 스트리밍 어플리케이션이 눈부시게 발전하고 있다[11]. 최근 가장 활발한 어플리케이션 개발 중 하나는 인터넷 방송 시스템이며 관공서와 금융업계를 중심으로 기존의 방송 시스템을 인터넷 방송 시스템으로 대체하는 작업이 활발히 이루어 지고 있다. 예를 들어 특정 구청에서 해당 동사무소로 전체 방송을 할 경우 기존에 국선을 이용한 방송 시스템은 유지비가 많이 들고 고품질의 오디오 방송은 불가능 하였다. 그러므로 기존의 방송 시스템을 인터넷 기반의 방송 시스템으로 구현 하여 보다 고품질의 오디오 방송을 저렴하게 사용 하고 있다. 일반적으로 방송 시스템 구축 시 좋은 성능을 발휘하나 오디오 스트리밍 시 다수의 수신측에 의한 혼잡 상황 발생 또는 버스트 트래픽이 발생하는 경우에는 128KHz 샘플링 이상의 고품질 오디오 방송 시 패킷 손실(packet loss)과 지연(delay)으로 인한 수신측 재생 음질 저하를 초래할 수 있다. 재생 음질 저하의 가장 주요한 원인은 네트워크 혼잡에 의한 패킷 손실이며 혼잡 제어가 가능한 부분은 송신측에서 혼잡제어 기법, 해당 네트워크 라우터 버퍼 관리 기법으로 나누어 볼 수 있다. 그러나 송신측의 혼잡제어 기법은 혼잡 발생 후 그것을 인지하고 전송측에서 데이터 전송량을 줄이는 방법으로 다소 수동적인 방법이

다. 이에 반해 라우터 버퍼 관리 기법은 네트워크 혼잡 시 라우터에 반입되는 패킷을 효과적으로 관리하여 혼잡을 막기 위한 능동적인 방법이다. 라우터 버퍼관리 기법으로는 평균 버퍼크기를 이용하여 혼잡을 미리 예측하여 확률적으로 패킷을 폐기하는 Random Early Detection(RED) 알고리즘이 대표적인 예이다[1]. 그러나 이 방법은 여러 종류의 모든 플로우(Flow)에 대해서 동일한 방법으로 패킷 폐기를 결정하므로 문제가 발생될 수 있다. 또한 TCP와 같이 전송 프로토콜 레벨에서 혼잡제어를 하는 프로토콜과 UDP 같이 혼잡제어를 하지 않는 프로토콜이 동시에 같은 링크를 공유하였을 때 공평하게 자원을 공유 할 수 없다는 문제가 대두된다. 이로 인해 실시간 스트리밍(Real-Time Streaming)에서 흔히 사용되는 UDP 프로토콜에서는 RED 알고리즘 적용은 적합하지 않다. 이를 극복하기 위하여 각 플로우 별로 패킷 폐기를 결정하는 Flow RED(FRED)가 등장하였다[2]. 이 방법은 다수의 플로우 연결 시 플로우 마다 상태 정보를 가지고 계산하기 때문에, 복잡성 증가라는 단점과 다수의 서로 다른 종류의 플로우가 동시에 연결 되었을 때 공정성이 깨지는 문제가 발생되고 이를 해결하기 위한 연구들이 계속 되고 있다.

제 2 절 연구 목적 및 제안 기법

인터넷 상의 TCP/UDP 플로우 사이의 대역폭 보장과 공정성 유지를 위한 방법으로는 FRED 라우터 버퍼 알고리즘을 사용하는 것이다. 이 알고리즘은 기존의 RED알고리즘이 갖고 있는 혼잡 제어를 하는 트래픽과 하지 않

는 트래픽 간의 공정성 문제를 해결 하였으나 RED과 같이 고정된 파라메터(Parameter) 사용으로 인한 성능 저하 문제와 각각의 플로우 마다 상태 정보를 관리해야 하므로 다수의 플로우로 인한 복잡성의 문제가 있다. 전자의 문제는 RED와 동일한 방법으로 Dynamic 파라메터를 사용하여 해결 하였고 후자의 문제는 가상의 논리큐를 두어 트래픽 종류별로 플로우를 관리하는 방법을 통해 해결 하였다. 그리고 다른 하나의 문제점은 다수의 TCP/UDP 플로우가 동일 링크를 공유 하였을 경우 TCP 플로우의 대역폭 점유율이 급격히 감소하는 현상이다[2][19]. 이런 문제에도 불구하고 현재 대부분 논문들의 시뮬레이션은 그리 많지 않는 수의 플로우 개수에 국한되어 있다.

본 논문에서는 다수의 TCP/UDP Flow가 동일한 링크를 공유하여 생기는 성능 저하 문제를 시뮬레이션을 통해 살펴 보고 이를 보안하기 위해 실험적으로 FRED를 확장한 Many-FRED 알고리즘을 제안 하고, 적용하여 시뮬레이션 후 그 결과를 토대로 기존 방법과 비교, 분석한다. 논문의 내용을 간략히 살펴보면 2장에서는 전송 프로토콜과 종단간 흐름제어 기법을 TCP 버전에 따라 설명하고 흐름제어를 하지 않는 트래픽의 전송률 제어 방법에 대해 논의 한다. 3장에서는 기존 라우터 버퍼관리 알고리즘인 RED에 대해 알아보고 RED의 확장 버전인 ARED에 대하여 설명한다[3]. 또한 FRED의 특성과 문제점에 대해 논의하며, 동일한 링크에 다수의 TCP/UDP 플로우 연결 시 발생 하는 FRED의 문제점을 살펴본다. 4장에서는 다수의 TCP/UDP Flow 환경에서 FRED 알고리즘의 many 옵션을 설정하여 TCP

버전에 따라 시뮬레이션 후 그 결과를 분석하며, “two packet buffer” 방식을 확장한 “specific random drop” 알고리즘을 제안하고 이를 통해 확장된 Many-FRED 알고리즘을 다수의 TCP/UDP Flow 환경에 실험적으로 적용하여 시뮬레이션을 하고, 그 결과를 토대로 기존 방법과의 성능을 비교, 분석한다. 마지막 5장에서는 본 논문의 주요 분석 결과에 대해 정리하고 향후 추가 연구 방향을 제시한다.

제 2 장 관련 연구

제 1 절 전송 프로토콜

본 절에서는 인터넷 프로토콜에서 전송 프로토콜로인 TCP, UDP를 살펴보고, TCP 버전에 따른 혼잡 제어 알고리즘에 대해 알아본다[6][11].

(1) TCP (Transmission Control Protocol)

TCP와 UDP는 서로 같은 네트워크 계층(IP)를 이용하지만, TCP는 흐름 제어, 연결지향의 신뢰성 있는 바이트 스트림 서비스를 제공한다. 흐름 제어란 데이터 송수신 시 송신자와 수신자가 적절하게 통신 속도를 조절하는 것을 말하며, 송신자는 수신자가 요구 하는 데이터 크기 이하로만 전송해야 한다. 이러한 데이터 전송 양을 제한하는 것이 흐름 제어이다. 연결지향형이란 TCP를 이용하는 종단 응용간 데이터를 교환하기 전에 서로 제어 패킷을 통해 핸드쉐이킹(Handshaking)을 하여 연결을 확립하는 방법이다. 그리고 신뢰적 데이터 전송은 한 응용이 그 연결을 통해 오류 없이 보내어진 순서대로 모든 데이터를 전달할 수 있다는 것을 의미한다. 인터넷에서의 신뢰성은 확인 응답(Acknowledgement)과 재전송에 의해 보장된다. TCP 연결은 전이중(full-duplex) 데이터 전송을 제공하며, 이는 각 종단에서 데이터 송신수신이 가능함을 말한다. 그러나 한 송신측에서 여러 수신측에게 데이터를 전송하는 멀티캐스팅은 불가능하다.

TCP 세그먼트 구조는 헤더 필드와 데이터 필드로 구성되어 있다. 데이터 필드는 응용 데이터를 포함하며 MSS(Maximum Segment Size)는 세그먼트의 데이터 필드의 크기를 제한한다. 초기 버전은 RFC793에 정의되어 있다 [4].

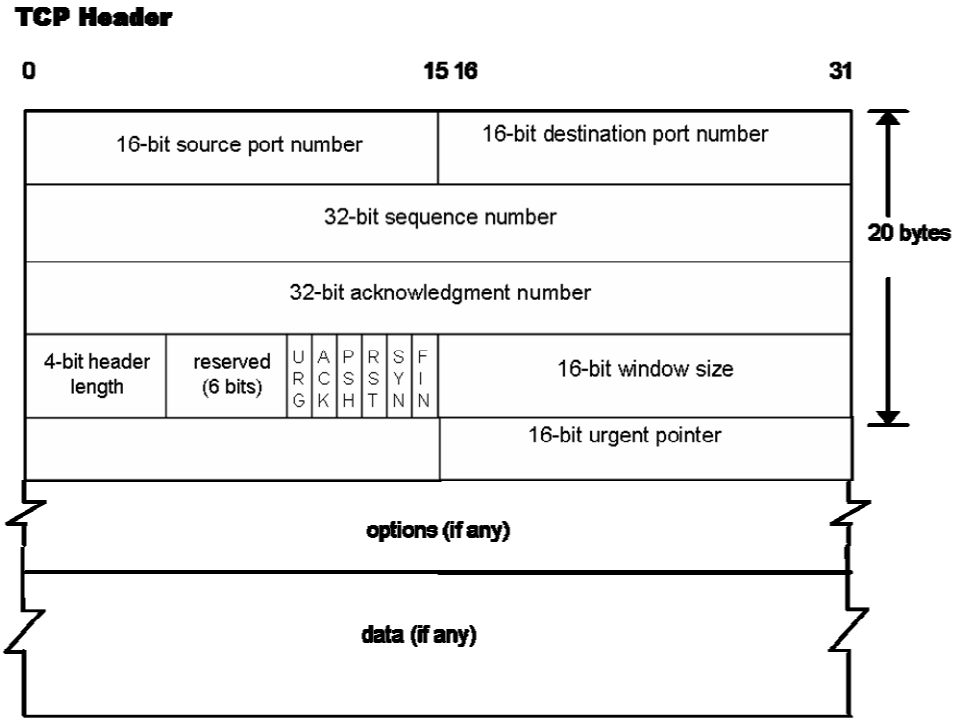

그림 2-1. TCP 세그먼트 구조

(2) UDP (User Datagram Protocol)

UDP는 단지 트랜스포트 계층 프로토콜이 할 수 있는 최소한의 기능으로 동작하며,다중화/역다중화 기능과 간단한 오류 검사 기능을 제외하곤 IP에 아무것도 추가 하지 않는 프로토콜이다. 즉, 응용 개발자가 TCP 대신에

UDP를 이용하면 응용은 거의 IP와 직접적인 통신을 하는 것이며, 응용 프로세스로부터 전송할 데이터를 가져와 서비스에 대한 소스 포트 번호 필드와 목적지 포트 번호 필드를 첨부하고 다른 두 필드들을 추가한 후 최종적인 세그먼트를 네트워크 계층으로 포워딩 한다. 네트워크 계층은 세그먼트를 IP 데이터그램으로 캡슐화하고, 세그먼트를 수신 호스트에게 최선형(Best-Effort) 서비스로 전송한다. 만약 세그먼트가 수신 호스트에 도착한다면, UDP는 세그먼트의 데이터를 해당하는 응용 프로세스에게 전달하기 위해서 목적지 포트 번호를 이용한다. UDP는 세그먼트를 송신하기 전에 송신측과 수신측 사이에서 핸드쉐이크를 통한 연결 확립을 않기 때문에 UDP를 비연결형 이라고 부른다. UDP 세그먼트 구조는 RFC 768에 정의 되어 있으며, 응용 데이터는 UDP 데이터 그램 데이터 필드에 위치한다. 예를 들어 스트리밍 오디오 응용에서는 오디오 샘플들이 데이터 필드를 채운다[5].

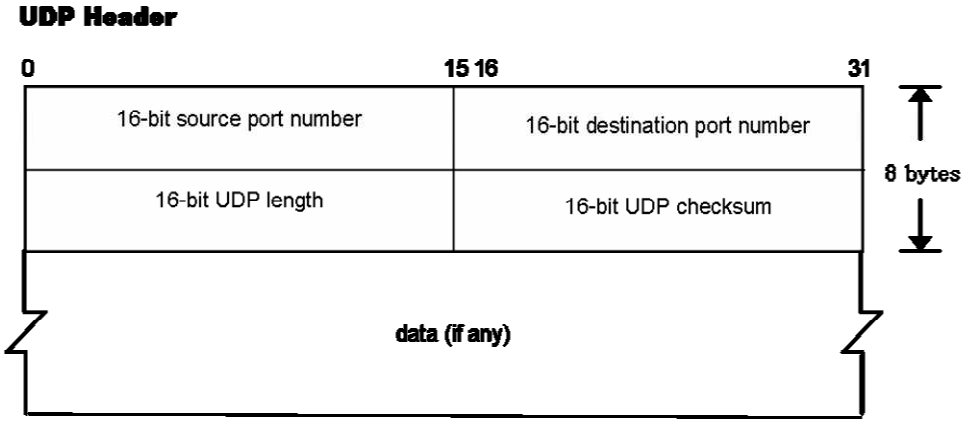

그림 2-2. UDP 세그먼트 구조

제 2 절 종단간 혼잡 제어

(1) TCP 혼잡 제어 기법

TCP/IP 네트워크의 종단간의 혼잡 제어는 TCP에 의해 수행 된다. UDP를 이용한 서비스는 혼잡 제어나 신뢰성 있는 전달이 보장 되지 않는다. TCP는 인터넷의 Best-Effort 특성으로 인하여 발생하는 비순서적 패킷 전달과 패킷 중복 전달, 손실 등의 문제에 대해 신뢰성 있는 데이터 전송을 위한 프로토콜이다. TCP의 혼잡 제어 방법은 네트워크 혼잡 상황에 따라 수신자에게 트래픽을 보내는 전송률을 각 송신자 제한 하도록 하는 것이다. 만약 TCP 송신자가 자신과 목적지 간의 경로에서 혼잡에 없음을 감지하면, 송신자는 전송률을 높인다. 만약 송신자가 경로 사이에 혼잡을 감지하면, 송신자는 전송률을 줄이는 방법을 사용 한다. TCP 혼잡 제어 방법은 저속 출발(Slow Start), 혼잡 회피(Congestion Avoidance), 빠른 재전송(Fast Retransmission)과 빠른 복구(Fast Recovery)등의 메커니즘을 혼용하여 구현 된다[7].

인터넷 상에서 데이터 송수신 시 패킷 손실이 발생하는 주요한 원인은 해당 경로에 있는 라우터가 버퍼 용량을 초과하여, 반입되는 패킷을 폐기 시키기 때문이다. 즉, 데이터 전송 에러로 인한 손실은 거의 없고 대부분이 라우터 버퍼의 용량 초과로 인해서 패킷이 손실 된다. 이러한 이유로 TCP는 패킷이 손실되어 발생하는 재전송 타임아웃을 해당 경로의 혼잡 상황으로 인지 하고 전송되는 데이터 양을 줄인다.

현재 TCP는 혼잡 윈도우 (Congestion Window) 크기를 결정하는 방법으로 AIMD(Additive Increase Multiplication Decrease)을 사용한다. 이 방법은 해당 네트워크가 혼잡 상황이 아닐 경우(수신자가 보낸 ACK 신호를 정상적으로 송신자가 받는 경우)에는 혼잡 윈도우 크기를 하나씩만 가수적(Additive)으로 증가 시키고 혼잡 상황 일 경우는 이전의 값의 반으로 지수적(Multiplication)으로 감소 시킨다. 이러한 방법을 통해 사용 가능한 대역폭을 효율적으로 송신자가 조절 하는 기법 이다.

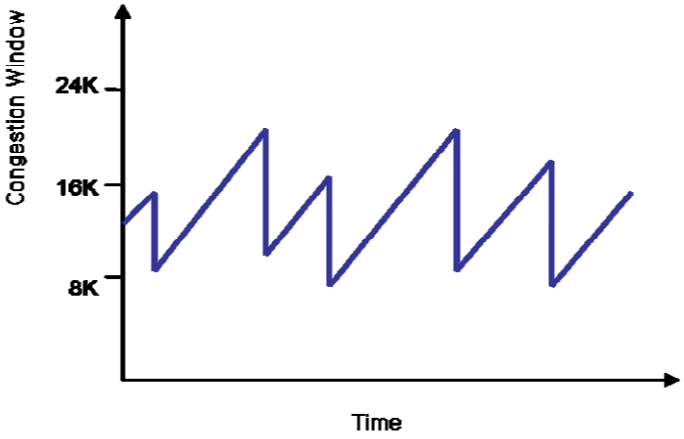

그림 2-3. AIMD 알고리즘 동작 예

a) Slow Start

Slow Start(저속 출발)는 TCP가 혼잡 상황을 해결하기 위해 사용하는 방법이다. 즉, 혼잡 상황을 인지하면 Cwnd(Congestion Window)를 1로 만들고 그 이후 지수적으로 증가 시켜 해당 네트워크의 가용 대역폭에 빠르게 접근하기 위해 사용하는 방법이다. Slow Start의 구현 방법은 송신측의 전

송 데이터의 Cwnd를 사용한다. 연결 초기의 Cwnd는 1로 설정되어 수신측으로부터 ACK를 받기 전까지는 단지 한 개의 패킷만 송신할 수 있다. 이후, 이 패킷에 대한 ACK를 받으면 Cwnd의 값을 2로 증가 시킨다. 그리고 송신측은 2개의 패킷을 연속해서 송신할 수 있고, 다음 ACK를 받으면 Cwnd 크기는 4가 된다. 이와 같이 응답을 받을 때마다 Cwnd의 값은 하나씩 증가하게 되며 Cwnd 크기를 Round-Trip Time 마다 지수적으로 빠르게 증가 시키게 된다. 이러한 Cwnd의 값은 실제로는 수신측이에서 송신측에게 수신측이 현재 남은 버퍼 크기를 알려 주는 Awnd(Advertised-Window)와 비교해서 최소값으로 설정된다. 이렇게 빠르게 허용 가능한 윈도우 크기 만큼 증가 시키는 방법을 Slow Start라 부르는 이유는 혼잡 발생이 전송 가능한 Awnd 만큼 전송 하는 것이 아니고 Cwnd를 1로 만들어 다시 처음부터 전송 데이터양을 증가 하기 때문이다. 또한 이 방법을 사용함으로써 버스트(Burst) 트래픽이 발생하지 않도록 한다.

TCP에서 Slow Start은 처음 연결 확립과 타임 아웃에 의한 재전송에만 국한 된다. Cwnd가 어느 이상 증가하면, 송신측은 링크의 전송 속도에 포화된 만큼의 패킷을 연속적으로 송신하게 된다. 실제 네트워크에서는 혼잡 제어에 의해서 패킷이 손실 되거나 오류가 생긴다. 이러한 경우 재전송 과정이 수행 되면서, 최대 Cwnd 크기가 반으로 줄게 되는 Congestion-Avoidance 단계로 진입한다.

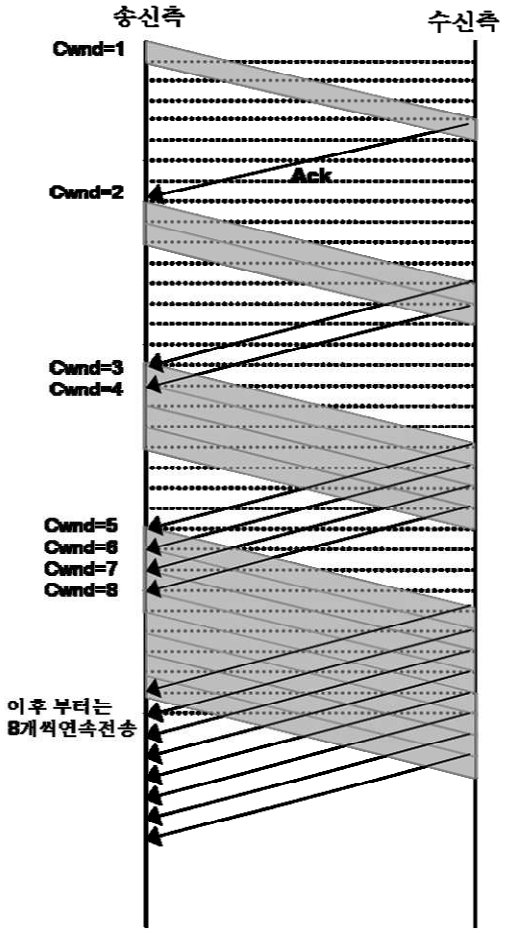

그림 2-4. TCP 에서 Slow Start 동작

b) Congestion Avoidance

Slow Start는 연결을 통해서 데이터 흐름을 초기화 하는 방법이다. 그러나 어떤 시점에서 해당 라우터의 버퍼 용량을 초과하면 패킷은 폐기 된다. 혼잡 회피는 이렇게 손실된 패킷 문제를 해결 하는 방법이다. 이 알고리즘의 전제는 패킷 전송 에러에 의한 손실은 매우 적다는 것이다. 그러므로 전송된 패킷의 주어진 시간동안 ACK 응답을 받지 못하는 패킷 손실의 경우 송

신측과 수신측 사이의 네트워크 어딘가에서 혼잡이 발생하고 있다는 것을 의미한다. 패킷 손실을 나타내는 것은 타임아웃 발생과 3개의 중복 ACK 수신의 2가지 경우이다. Slow Start와 Congestion Avoidance는 목적이 서로 다른 독립적인 알고리즘이다. 그러나 혼잡 상황을 인지할 경우 송신측에서 네트워크로 전송하는 패킷 전송률을 감소하기 위해 Slow Start를 이용하게 된다. 실질적으로 이 2개의 알고리즘은 함께 사용된다. 이 두 알고리즘을 적절히 적용 하기 위해서는 Cwnd와 Slow Start의 임계값(ssthresh)을 이용한다. 만약 Cwnd가 ssthresh보다 작거나 같다면, Slow Start 알고리즘을 사용하며, 그렇지 않으면 Congestion Avoidance 알고리즘을 사용한다. 이 단계에서는 Slow Start 단계에서와 같이 Cwnd을 지수적으로 증가 시키는 것이 아니라 가수적으로 증가 시킨다. 이는 전송 증가율을 줄여 전송 가능한 데이터량을 최대로 하기 위함이다.

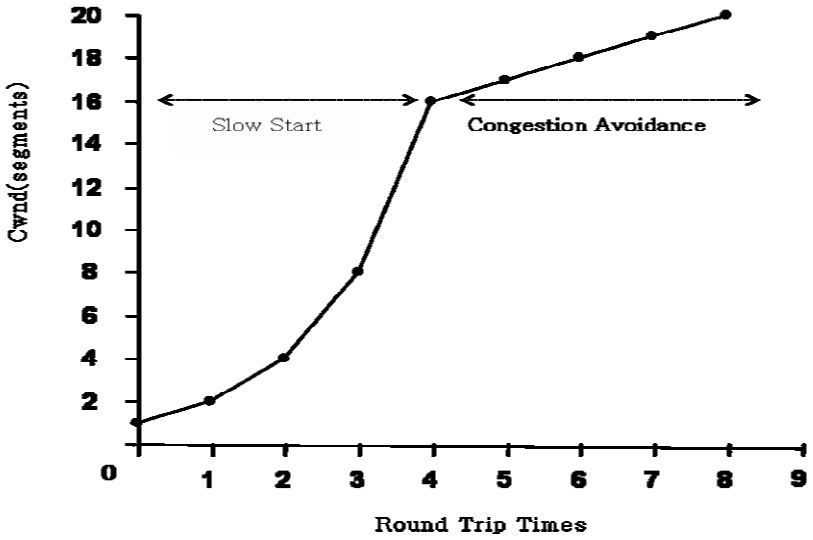

그림 2-5. RTT에 따른 혼잡 윈도우 변화

c) Fast Retransmission

TCP 혼잡제어는 패킷 손실을 인지 하기 위해서는 타임 아웃 동안 기다려야 하므로 매우 불합리하다. 이를 극복하기 위해서 Fast Retransmission이 사용된다. TCP에서 수신측에서 수신되어야 할 세그먼트 순서번호와는 다른 순서번호를 가진 세그먼트를 수신 할 경우를 위해서 중복 ACK(Duplicate ACK)를 송신측에 전송 한다. 중복 ACK의 목적은 세그먼트 번호가 잘못 보내졌다는 것과 기대하고 있는 순서 번호가 무엇이라는 것을 송신측에 알리는 것이다. 중복 ACK가 손실 세그먼트에 의한 것인지 단지 순서번호 오류에 의한 세그먼트 교체에 의한 것인지 모르기 때문에 몇 개의 중복 ACK를 기다려야 한다. 즉, 하나 또는 두 개의 중복 ACK에 대해서는 세그먼트 순서가 잘못된 것으로 간주하며 보통 3개 이상의 중복 ACK는 패킷 손실에 의한 행동으로 간주하고 이 경우 타임 아웃까지 기다리지 않고 재전송을 하게 되는데, 이를 Fast Retransmission라고 한다. 이 방법은 대략적으로 TCP 타임 아웃을 1/2로 줄여 20% 정도의 성능 향상을 가져온다. 그러나 이 방법 역시 혼잡 윈도우 크기가 매우 적은 경우 3개의 중복 ACK를 수신할 수 없으므로 타임 아웃까지 기다렸다가 재전송 한다.

d) Fast Recovery

Fast Recovery는 3개의 중복 ACK의한 Fast Retransmission 경우 Slow Start 단계로 진입하지 않고 바로 Congestion Avoidance 단계로 전환되는 것을 말한다. 이는 Slow Start 인한 급격한 전송률 저하를 막기 위함이다. TCP는 3개의 중복 ACK를 수신 할 경우 혼잡에 의한 패킷 손실로 인지하

고 현재 윈도우 크기 (Cwnd과 Awnd 중 최소값)의 1/2로 ssthresh로 설정한다. 그리고 다음 다음 Cwnd는 ssthresh+3으로 설정한다. 만약 네 번째, 다섯 번째 등 이후에도 중복 ACK가 도착하면 Cwnd를 1씩 증가 시킨다. 이후, 도착하는 ACK가 새로운 패킷에 대한 것이라면, Cwnd를 ssthresh로 설정한다. 이 방법은 패킷 손실 후 다시 Slow Start 단계를 거치지 않고 바로 ssthresh 값부터 Congestion Avoidance 단계로 전환되어 윈도우 크기를 빠르게 회복시켜 전송율을 증가시키기 위한 기법이다.

(2) TCP 버전에 따른 혼잡 제어 기법

TCP는 혼잡 제어 기법에 따라 현재 여러 종류의 TCP 버전이 구현 되어 있으며 주요한 TCP 버전은 아래와 같다[12].

a) TCP Tahoe

TCP/Tahoe는 혼잡 제어 시 Slow Start, Congestion Avoidance, Fast-Retransmission 메커니즘을 포함한 알고리즘이다. Tahoe의 경우 패킷 전송 시작 시 Slow Start 단계로 접어 들며, 혼잡 윈도우가 임계치를 초과 하면 Congestion Avoidance 모드로 동작한다. 그리고 패킷 손실이 발생 하면 무조건 혼잡 윈도우(Cwnd)를 1로 줄이고, 또 다시 Slow Start 단계로 진입한다. 이때 임계치값은 이전 임계치(ssthresh)의 1/2로 감소 시킨다. 이와 같이 Tahoe 의 경우 패킷 손실 감지 후 무조건 Slow Start로 동작하며, Fast Recovery는 지원 되지 않는다. 즉, TCP/Tahoe의 단점은 패킷 손실 시 무조건 타임아웃까지 기다려야 하는 문제가 있다.

b) TCP Reno

TCP/Reno 버전에서는 Fast Recovery 알고리즘이 추가되었다. 즉, 중복 ACK에 수신에 따라 Fast Retransmission 하고 Cwnd를 1로 설정하는 Slow Start 단계로 진입 하지 않고 바로 윈도우 크기를 현재 윈도우 크기의 절반으로 줄이고 Congestion Avoidance 모드로 전환 되도록 하였다. TCP/Reno의 경우 초기 TCP 연결 시에는 TCP/Tahoe와 같은 방법으로 처음에는 Cwnd를 1로 설정하고 그 후 지수적으로 증가 시키는 Slow Start 단계로 동작하며 임계치 초과 시에는 Congestion Avoidance 단계로 전환 된다. 그러나 TCP/Tahoe와의 차이점은 3개의 중복 ACK에 대한 패킷 손실과 타임아웃에 의한 패킷 손실을 다른 방법으로 처리한다. 먼저 재전송 타임아웃에 의해 패킷 손실을 인지 했을 경우 Fast Retransmission을 거쳐 Cwnd를 1로 설정하여 Slow Start 단계로 진입해서 동작하고, 3개의 중복 ACK에 의한 패킷 손실을 인지 했을 경우는 Fast Retransmission을 거쳐 현재 윈도우 크기를 이전 윈도우 크기의 1/2로 줄여 Fast Recovery 모드로 전환되어 Congestion Avoidance 기법을 통해 이후 Cwnd를 가수적으로 증가 시킨다. 즉 패킷 손실을 감지 했을 때 재전송 후 최적의 윈도우 크기에 보다 빨리 도달하기 위하기 위함이다. TCP/Reno의 Fast Recovery 알고리즘은 해당 윈도우 내에서 하나의 패킷 손실에 대해서만 최적의 성능을 발휘한다. 그러나 다수의 패킷이 연속적으로 손실되었을 경우에는 혼잡 윈도우 크기가 지수적으로 감소하고 가수적인 증가에 의해서 오히려 TCP/Tahoe보다 성능이 떨어지게 되는 문제가 있다.

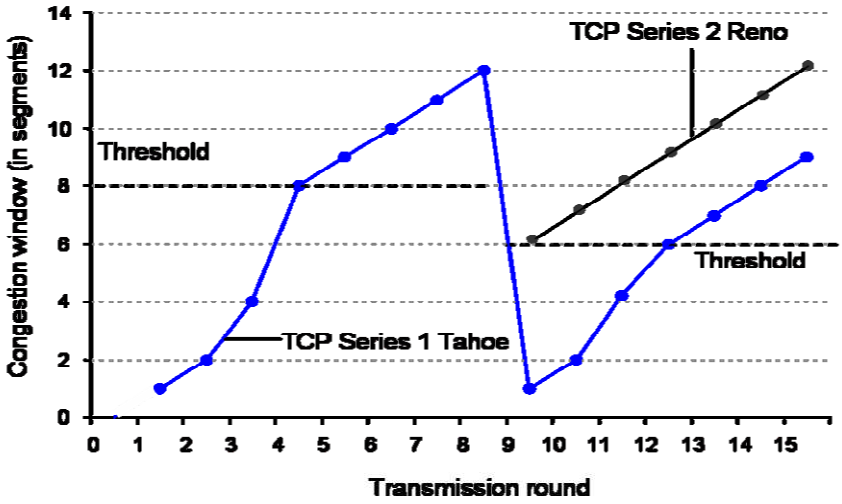

그림 2-6. TCP/Tahoe와 TCP/Reno 환경에서의 혼잡 윈도우 변화

c) TCP NewReno

TCP/Reno 버전에서는 만약 두 개 또는 그 이상의 패킷이 같은 전송 윈도우 내에서 손실될 경우 Fast Retransmission과 Fast Recovery 알고리즘도 재전송 타임아웃을 기다리지 않고서는 여러 손실들로부터 회복 할 수 없다. 이런 경우 Reno는 타임아웃에 의한 성능 저하가 발생 한다. 그러나 TCP/NewReno 버전은 기존의 Fast Recovery 기법을 수정하여 여러 개의 패킷 손실이 발생하더라도 타임아웃 없이 복구할 수 있는 알고리즘이다. NewReno에서는 한 개의 윈도우내에서 여러 개의 패킷 손실이 발생하면 손실된 패킷에 대해서만 수신측은 Partial ACK를 송신측에 전송하며, 이를 받은 송신측은 해당 패킷을 재전송 한다. 이 방법에 의해 패킷을 전송하고 손실된 패킷이 모두 재전송이 완료되면 수신측에서는 이를 알리는 positive ACK를 송신측으로 전송하고 이를 받은 송신측은 Cwnd값을 이전의 Cwnd

값으로 설정하고 데이터를 전송한다. 이로 인해 한 윈도우내에 여러 개의 패킷 손실이 발생하더라도 타임 아웃에 걸리지 않는다는 장점이 있다. 그러나 종단간 모두 TCP/NewReno를 사용해야만 하는 단점이 있다.

d) TCP Vegas

TCP/Vegas는 Tahoe 와 Reno가 패킷 손실을 인지 하지 못하는 동안은 Cwnd를 계속 증가시키는 반면에 Vegas는 전송한 패킷 들의 왕복시간(RTT)를 측정하여 혼잡 윈도우 크기를 증가시키거나 또는 감소 시킨다. TCP/Vegas는 Slow Start, Fast Retransmission 알고리즘과 Congestion Avoidance 알고리즘을 변형해서 구현 된다. Fast Retransmission의 수정된 부분은 패킷 손실의 원인이 되는 3개의 중복 ACK를 통해서 패킷 손실을 인지 하는 것이 아니라 중복 ACK에 대해 무조건 패킷 손실로 간주한다. 이는 TCP/Reno에 타임 아웃 발생으로 허비되는 시간을 절반 가량 줄일 수 있다. 또한 수정된 Congestion Avoidance 기법은 혼잡 상황이 아닌 상태에서 BaseRTT값을 구하고 BaseRTT를 기반으로 하는 기대되는 전달률 ExpectedRate(=Cwnd/BaseRTT)를 계산한다. 그리고 패킷을 전송하고 ACK 응답 시간을 계산하여 실제 전달률(ActualRate)를 구한다. 그리고 ActualRate와 ExpetedRate의 차이를 이용해 윈도우 크기를 변화 한다. 마지막으로 수정된 Slow Start 기법을 통해 RTT마다 Cwnd를 지수적으로 증가하도록 하는 방법을 이용하여 성능 향상을 꾀하였다. 그러나 상대방이 사용하는 TCP 버전이 TCP/Reno인 경우 대역폭을 공정하게 공유하지 못하는 단점이 존재하고 이 버전을 사용하는 운영체제가 없다는 것이 문제점

이 된다.

e) TCP SACK

이 버전 역시 TCP/NewReno와 같이 이전의 TCP 알고리즘은 동일한 윈도우내에 여러 개의 패킷이 손실될 경우에 TCP 성능저하 문제를 해결 하고자 하는 알고리즘이다. TCP/SACK (Selective Acknowledgement)는 동일한 윈도우에서 여러 개의 패킷이 손실된 경우 TCP 옵션 필드의 SACK을 사용하게 되면 송신측은 손실된 패킷에 대해서만 다시 전송하고, 수신측은 성공적으로 도착한 모든 패킷에 대해 송신측에 알려 준다. 이로 인해 한 윈도우내에 여러 개의 패킷 손실이 발생하더라도 타임 아웃에 걸리지 않는다는 장점이 있다. 그러나 종단간에 TCP SACK 옵션 필드가 enabled 되어 있지 않는다면 TCP/Reno 모드로 작동되는 문제가 있다[12].

(3) TFRC (TCP-Friendly Rate Control)

해당 네트워크가 혼잡 상황 시 흐름 제어가 가능한 TCP와 그렇지 않는 UDP가 동일 링크를 공유 하였을 경우 TCP의 혼잡 제어 특징에 의해서 혼잡 제어를 하지 않는 UDP가 대부분의 대역폭을 점유하는 불공정한 현상이 일어난다. 이 문제를 전송 프로토콜 레벨에서 해결 하려는 메커니즘이 바로 TFRC (TCP-Friendly Rate Control) 이다. 이 기법은 흐름 제어를 하지 않는 트래픽에 대해서 TCP 혼잡 제어와 비슷하게 전송측에서 데이터 전송 속도를 조절하는 방법이며, 주로 실시간 멀티미디어 전송 프로토콜인 RTP(Real-time Transfer Protocol)를 사용하는 응용에 많이 사용 된다.

TFRC은 방정식 기반이며, 송신자가 측정된 loss event rate와 RTT를 가지고 전송률을 결정하여, 이용 가능한 대역폭을 사용하는 것이 아니라 TCP와 공정성을 유지하며, 꾸준한 전송 상태를 통하여 혼잡 상황에 적응하는 것이 설계 목표이며 그 방법은 다음과 같다[16].

• 전송률은 서서히 증가 시키고 loss event rate에 맞춰 감소한다. 즉, 공격적으로 사용 가능한 대역폭을 점유하지 않는다.
• 연속적인 loss event에 대해서는 전송률은 반으로 줄인다. 그러나 하나의 loss event에 대해서 전송률을 반으로 줄이지 않는다.
• 만약 일정기간 동안 수신자가 패킷을 수신 받는다면 적어도 매 RTT마다 한번씩 수신자에게 feedback 정보를 보고해야 한다.
• 만약 송신자가 여러 번의 RTT이후에도 수신자로부터 feedback을 받지 못하면 송신자는 전송률을 감소시키고 결국 전송을 멈춘다.

TCP의 전송률을 나타내는 방정식은 다음과 같으며 이를 혼잡 제어 알고리즘으로 사용한다.
• 파라메터: T (전송률), s (패킷 크기), RTT (Round-Trip Time), RTO (재전송 타임아웃), p (패킷 손실률)

$$T = \frac{s}{RTT\sqrt{\frac{2p}{3}} + RTOmin(1, 3\sqrt{\frac{3p}{8}})p(1+32p^2)}$$

그림 2-7. TFRC에서 사용되는 방정식

TCP 반응식을 혼잡 제어 알고리즘으로 이용하기 위해서는 다음과 같은 사항이 요구 된다.

• 파라메터 RTT와 p가 결정되어야만 한다. loss event rate p는 반드시 수신측에서 계산되어야 하고, RTT는 수신측 혹은 송신측 둘 중 한 곳에서 계산되어야 한다.
• 수신측은 가장 최근에 도착한 패킷의 시퀀스 번호, 받은 시간, 파라메터 p를 송신측으로 feedback 해야 한다.
• 송신측은 RTT, RTO, p값을 구해 혼잡제어 방정식을 이용해 TCP-friendly 전송률 값 T에 따라서 전송률을 늘리거나 줄인다.

이와 같이 계산식에 따라 동작하는 TFRC 알고리즘은 송신측, 수신측에서 파라메터를 계산하여 갱신하여야 하므로 다소 복잡도가 높은 방법이다.

제 3 장 FRED 알고리즘 분석 및 확장

제 1 절 기존 라우터 버퍼관리 알고리즘

본 절에서는 FRED 알고리즘의 장단점 분석하기 위해 기존 라우터 버퍼 알고리즘에 대해 먼저 설명한다.

(1) DT (Drop Tail)

TailDrop은 전통적인 라우터 큐 관리 방법으로 단순히 큐에 들어오는 순서대로 버퍼에 저장 한다. 그러다가 버퍼 오버플로우(overflow)시에는 큐의 끝부분에 새로 들어오는 패킷 부터 폐기시키는 기법이다. 그러나 이 방법은 구현이 간단하다는 장점이 있으나 전체적으로 TCP 혼잡제어 윈도우 크기가 줄어들어 링크 효율이 저하되는 Global Synchronization 현상이나 높은 큐잉 지연 등이 문제점으로 나타난다[14].

(2) ERD (Early Random Drop)

ERD 방식은 기존의 수동적인 방법의 버퍼관리 기법(TailDrop)보다 능동적인 방법이다. 이 알고리즘은 해당 라우터 버퍼 크기에 대해 임계치를 설정하고 현재 버퍼 크기가 이 값을 초과하면 혼잡 상황이 발생 할 것이라 판단 하고 확률(Probability)값에 의해 반입되는 패킷을 폐기하여 버퍼 오버플로우를 미리 예방하며 송신측에 혼잡 상황 발생 가능성을 알린다. ERD는 기존의 TailDrop에 비해 더 빨리 혼잡 상황을 인지하여 패킷을 폐기 함으

로써 혼잡 상황으로부터 혼잡 제어를 하는 시점까지의 시간을 단축 시켰다.

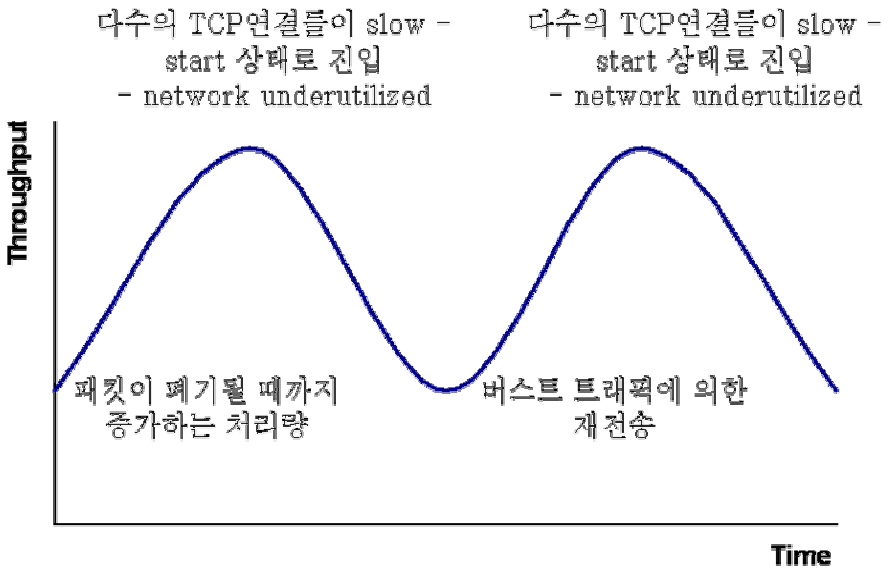

그림 3-1. 전역 동기화 (Global Synchronization)

(3) RED (Random Early Detection)

RED는 라우터에서 능동적 큐 관리에 기반을 두고 동작하는 혼잡회피 메커니즘이다. 이 방법은 TailDrop의 문제점을 보완하기 위한 목적으로 제안되었다[14]. RED 라우터는 버퍼공간이 오버플로우 되기 전에 미리 확률적으로 패킷을 폐기함으로써 TCP에 초기 혼잡의 신호를 보낸다. 이러한 확률적 패킷 폐기는 평균 큐 크기에 의해 동작함으로써 버스트 트래픽(Burst-Traffic)에 대해 민감하게 반응하지 않게 한다. RED 알고리즘은 미리 혼잡 상황을 예견하는 지표로써 버퍼 크기의 최소/최대 임계치를 정하여 놓고 라우터에 반입되는 패킷에 대해 EWMA(Exponentially Weighted Moving

Average)라는 계산식을 적용 하여 평균 큐 크기를 계산하여 이 값에 따라 임계치와 비교하여 패킷 폐기 여부를 결정하여 미리 혼잡 상황을 예방하는 효과적인 라우터 버퍼관리 기법이다.

RED는 두 개의 임계치 최소값(Minimum Threshold)과 최대값(Maximum Threshold)을 유지하고 평균 큐 크기에 의해 패킷의 큐잉 여부를 결정한다. 또한 패킷 폐기는 평균 큐 크기의 함수 p_a에 의해 확률적으로 이루어진다. 만일 패킷 반입 시 평균 큐 크기가 최소 임계치 보다 아래에 있으면 패킷 폐기 확률은 0%가 되어 모든 패킷은 큐에 반입된다. 그리고 평균 큐 크기가 최대 임계치와 최소 임계치 사이에서 변화하는 경우 p_a는 정해진 최대 폐기확률(max_p)에 의해 선형적(linearly)으로 변화하며 이에 따라서 패킷을 폐기한다. 또 다른 경우 평균 큐 크기가 최대 임계치 위에 있다면 폐기 확률은 100%로 도착하는 모든 패킷은 폐기된다. 이러한 방법에 의해 TCP의 혼잡을 미리 예측할 수 있고 버스트 트래픽으로 인한 전역 동기화(Global Synchronization)를 피할 수 있다.

RED는 특정 프로토콜에서 한정되어 사용되지는 않지만, TCP 프로토콜에서 보다 잘 수행되는데 이는 TCP는 패킷 폐기가 일어나면, 혼잡 상황이 발행 한 것으로 인식하고 TCP 프로토콜 레벨에서 혼잡제어를 하기 때문이다. RED알고리즘은 평균 큐 크기를 추정하는 부분과 반입되는 패킷을 폐기할지를 결정하는 부분의 두 개의 주요 부분으로 구성되어 있다.

a) 평균 큐 크기 추정

첫째로, RED 에서 평균 큐 크기 추정은 Exponentially Weighted Moving Average 또는 이와 유사한 메커니즘을 사용한다

$$avg \leftarrow (1 - w_q) * avg + w_q * q_{size}$$

표 3-1. 평균 큐 크기 계산식 (RED)

avg	EWMA에 의해 계산된 평균 큐 크기
w_q	avg를 결정하기 위한 가중치 ($0 < w_q < 1$)
q_{size}	최대로 수용 가능한 실제 큐 크기

표 3-2. 평균 버퍼 크기 계산에 사용되는 파라메터 정의 (RED)

b) 패킷 폐기 결정

둘째로, 패킷 폐기 결정은 라우터로 반입되는 패킷을 폐기 할 것인지 큐에 저장 할지를 결정하는 것이다. 이 부분은 패킷 폐기를 통하여 대화형 플로우들의 성능 향상을 위한 RED 의 중요한 알고리즘이다.

이 과정에는 중요한 두 개의 RED 파라메터 min_{th}(Minimum Threshold)와 max_{th}(Maximum Threshold)가 있다. 만약 평균 큐 크기가 min_{th} 보다 작으면 어떤 패킷도 폐기하지 않고 max_{th} 보다 크면 모든 패킷을 폐기한다. 그리고 평균 큐 크기가 min_{th}~max_{th} 사이에 있으면 확률에 의해 0~max_p 사이에서 선형적으로 폐기한다[1].

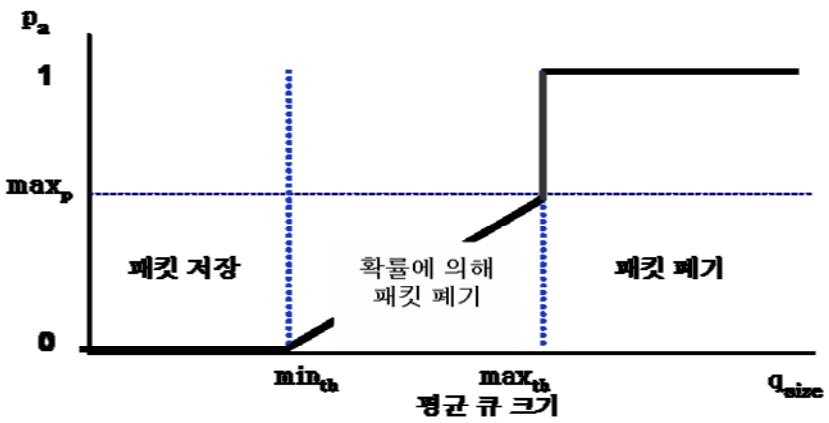

그림 3-2. RED 알고리즘의 패킷 폐기 확률 변화

p_b	계산에 사용되는 임시 확률
max_p	p_b의 최대값
avg	평균 큐 크기
min_{th}	패킷의 폐기를 결정하는 최소 임계 값
max_{th}	패킷의 폐기를 결정하는 최대 임계 값
p_a	현재의 패킷 폐기 확률
count	마지막 폐기된 패킷 이후의 패킷 수

표 3-3. 패킷 폐기 확률에 사용되는 파라메터 (RED)

$$p_b \leftarrow max_p * (avg - min_{th}) / (max_{th} - min_{th})$$

$$p_a \leftarrow p_b / (1 - count * p_b)$$

표 3-4. 패킷 폐기 확률값 계산식 (RED)

```
for each packet arrival
        calculate the new average queue size avg :
                if the queue is nonempty
                        avg ← (1 - w_q) * avg + w_q * q_size
                else
                        m ← f(time - q_time)
                        avg ← (1 - w_q)^m * avg
        if  min_th ≤ avg < max_th
                //increment count
                //calculate probability p_a :
                p_b ← max_p * (avg - min_th) / (max_th - min_th)
                p_a ← p_b / (1 – count * p_b)
                with probability p_a :
                mark the arriving packet
                count ← 0
        else if max_th ≤ avg
                mark the arriving packet
                count ← 0
        else count ← -1
       when queue become empty
           q_time ← time
```

그림 3-3. RED 알고리즘 의사 코드

time	현재 시간
Q_time	queue idle 시작 시간
f(t)	시간 t의 선형 함수

표 3-5. 그 밖의 시간 관련 파라메터 (RED)

(4) ARED (Adaptive Random Early Detection)

ARED 알고리즘은 모든 상황에 동일 고정 파라메터 값을 사용 하는 RED 와는 달리 라우터로 반입되는 트래픽에 따라 RED 파라메터 값을 동적으로 조절하여 최적의 성능을 발휘 하도록 제안된 방법이다[3].

ARED는 RED에서 성능에 주요한 역할을 하는 파라메터들 중에서 가장 성능에 영향을 미치는 폐기 확률 값을 위한 파라메터인 P_{max} 값을 현재 네트워크 트래픽 양에 따라 동적으로 관리하는 방법으로 Dynamic RED라고도 부른다. 즉, 해당 라우터에 반입되는 트래픽량이 많을 때는 P_{max} 값을 높여서 많은양의 패킷을 폐기하고, 반입되는 트래픽이 적을 때는 P_{max} 값을 낮춰서 적은양의 패킷을 폐기하여 효과적으로 버퍼를 관리 하는 방법이다. 그러나 P_{max} 값이 너무 높은 경우에는 많은 수의 패킷 폐기에 의해 전체 네트워크 성능이 감소하며, 반대의 경우 혼잡 상황이 오랜 기간 지속 되어 큐잉 딜레이가 증가 하게 된다.

제 2 절 FRED (Flow Random Early Detection)

본 절에서는 기존 라우터 버퍼 알고리즘의 문제점을 설명하며, FRED 알고리즘에 대해 설명, 비교하여 장 단점을 제시 한다. 또한 시뮬레이션을 통해 문제점을 확인한다.

(1) 기존 알고리즘의 문제점

앞서 1절에서 설명한 라우터 버퍼 관리 알고리즘인 RED는 혼잡을 미리 예측 하여 효율적으로 버퍼를 관리하는 알고리즘이다. 그러나 RED 알고리즘은 크게 두 가지 문제점을 내포하고 있다. 첫째, 모든 종류의 트래픽에 같은 고정된 파라메터 값을 사용하여 성능 저하의 문제가 있다. 둘째, 전송 프로토콜 레벨에서 혼잡 제어를 하는 TCP트래픽과 하지 않는 UDP트래픽이 동시에 같은 링크를 공유 하였을 경우 TCP의 혼잡 제어에 의해 UDP트래픽이 공유 링크의 대부분을 차지 하여 공정성에 문제가 생긴다. 전자는 고정 파라메터 중에서 가장 성능에 영향을 미치는 최대 패킷 폐기확률(max_p)를 상황에 맞게 동적으로 변화 시켜 성능을 향상 시키기 위해 ARED(Adaptive RED)알고리즘이 제안 되었고, 후자는 각 플로우 단위로 패킷 폐기 여부를 결정하는 FRED(Flow RED) 알고리즘에 의해 서로 다른 종류의 트래픽 사이의 공정성 문제를 해결하고자 하였다.

그림 3-4는 이러한 RED 알고리즘의 문제점을 확인하기 위한 네트워크 구성을 나타낸 것이다. 여기서 두 개의 TCP 플로우는 FTP Application을 사

용하였고, 두 개의 UDP 플로우는 CBR Application을 사용하였다. 또한 혼잡 상황을 유발 시키기 위하여 혼잡 구간의 대역폭은 그렇지 않은 곳은 1/10로 설정 하였다. RED, FRED 알고리즘 설정 값 들은 기본값으로 설정 하였으며, 시뮬레이션 시 사용되는 중요 파라메터는 많은 실험에서 사용되는 권고 값으로 설정 하였다(max_{th}=8, min_{th}=4, max_p=0.02, w_q=0.002, min_q=2, q_{size} = 50).

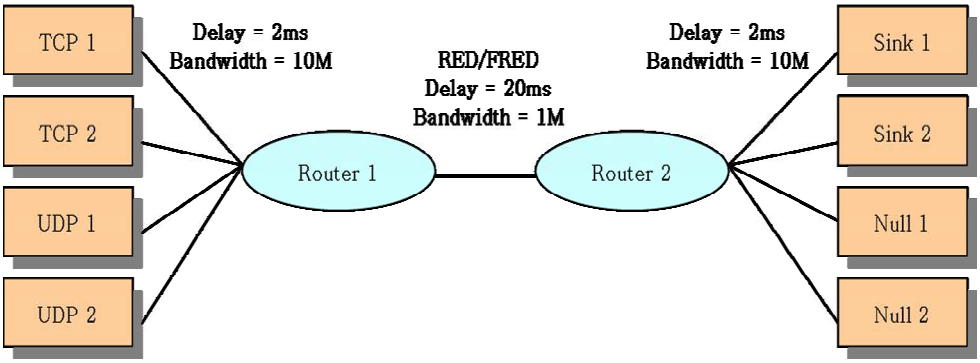

그림 3-4. TCP/UDP 플로우 연결을 위한 토폴로지

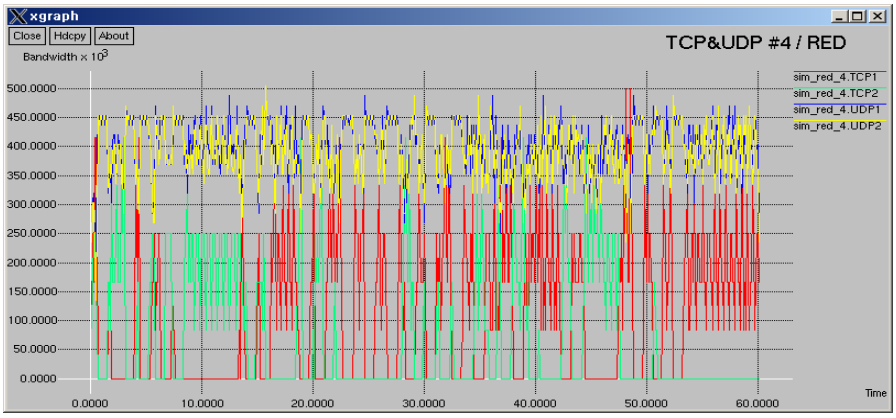

그림 3-5. TCP/UDP 플로우에 대한 대역폭 비교 (RED)

그림 3-5은 RED 알고리즘을 적용한 결과이다. 이와 같이 흐름 제어를 하지 않는 UDP가 더욱 많은 대역폭을 차지 하는 것을 알 수 있고 TCP 플로우 경우 패킷 손실로 인한 타임 아웃에 걸려 Slow Start 단계로 진입하는 경우가 종종 발생하는 것을 알 수 있다.

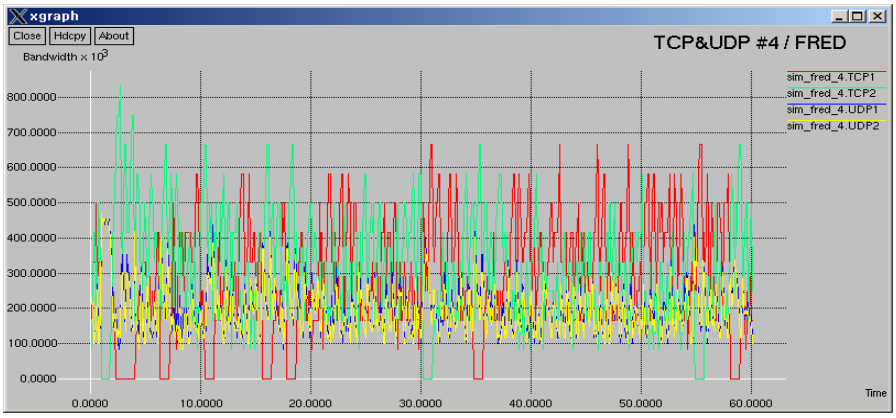

그림 3-6. TCP/UDP 플로우에 대한 대역폭 비교 (FRED)

	RED				
	TCP 1	TCP 2	UDP 1	UDP 2	전체
대역폭 점유율	10.74%	7.16%	39.79%	39.31%	97.00%

표 3-6. 평균 대역폭 점유율 (RED)

	FRED				
	TCP 1	TCP 2	UDP 1	UDP 2	전체
대역폭 점유율	28.41%	32.05%	19.72%	18.92%	99.10%

표 3-7. 평균 대역폭 점유율 (FRED)

	RED			
	TCP 분산	TCP 표준편차	UDP 분산	UDP 표준편차
MBPS	0.0003	0.0179	0.0000	0.0024

표 3-8. 분산 및 표준편차 (RED)

	FRED			
	TCP 분산	TCP 표준편차	UDP 분산	UDP 표준편차
MBPS	0.0003	0.0182	0.0000	0.0040

표 3-9. 분산 및 표준편차 (RED)

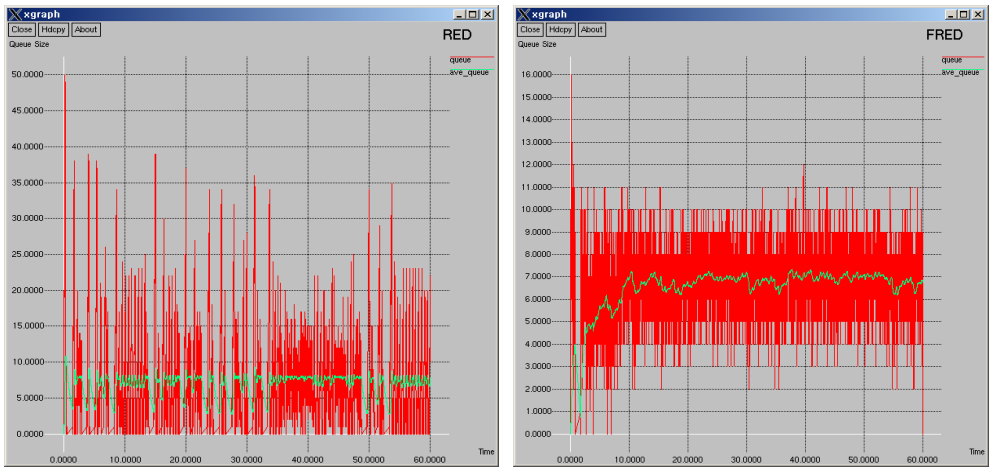

그림 3-7. RED와 FRED의 큐 점유 상태 비교

위의 시뮬레이션 결과를 통해 혼잡 제어를 하는 TCP 플로우와 하지 않는 UDP 플로우간의 대역폭 점유 정도를 RED와 FRED를 적용하여 알아보았다. 표 3-8, 3-9에서는 대역폭, 분산 정도를 Mbps 단위로 나타내었다. 그

림 3-5, 3-6과 표 3-6, 3-7를 통하여 두 RED, FRED 알고리즘의 차이점을 알 수 있다. RED 알고리즘 적용 시 79.10%에 육박하던 UDP 플로우들의 대역폭 점유율은 FRED 알고리즘 적용으로 38.64% 정도로 줄어 들었고, RED 알고리즘 적용 시 17.90%정도에 불과하던 TCP 플로우의 점유율이 FRED 알고리즘 사용으로 60.46% 가까이 상향 조정 된 것을 알 수 있다. 또한 그림 3-7와 같이 RED 보다 FRED가 큐 점유 상태도 보다 더 안정적이다. 그러나 FRED에서의 UDP 표준 편차가 약간 증가하는 것을 알 수 있다. 이러한 결과는 파라메터 설정과 UDP 전송 데이터 양에 따라 다소 변할 수 있다. 위의 시뮬레이션 결과를 토대로 이기종의 트래픽간에 공정성 향상을 위해서는 RED 보다 FRED가 더 효과적으로 공유 대역폭을 공평하게 나누어 쓰는 알고리즘이라는 것을 확인 할 수 있다.

(2) FRED 알고리즘 분석

FRED(Flow Random Early Detection) 알고리즘은 RED 알고리즘의 혼잡 제어를 하는 TCP 플로우가 혼잡 제어를 하지 않는 UDP 플로우와 동일한 링크를 공유 하였을 때 발생 되는 불공정성을 극복하기 위해 제안 되었다 [2]. 이 메커니즘은 현재 라우터 버퍼에 누적되어 있는 패킷의 플로우를 구분하여 각 플로우 마다 처리하여 공정성 개선을 한 알고리즘이다. FRED는 RED와 동일한 방법으로 단일 FIFO(First In First Out) 큐를 사용하나, 전체 플로우를 대상으로 평균 버퍼 크기를 계산하여 폐기를 결정짓는 RED와는 달리 서로 다른 종류의 플로우들 기반에서 공정한 대역폭을 공유하기 위하여 각 플로우 마다 패킷 폐기를 결정하여 공정한 대역폭 분배가 이루

어 지도록 하는 기법이다.

FRED 알고리즘 역시 기본적인 개념은 RED와 같다. 즉, 각 플로우에 허용 가능한 범위를 두 개의 최소 임계치와 최대 임계치를 유지하고 이를 초과하는 경우 적절히 패킷을 폐기하여 버퍼관리를 한다. 그리고 패킷이 반입될 때 플로우별로 상태 정보를 관리하기 위한 파라메터인 Nactive, avgcq, min_q, $qlen_i$, $strike_i$ 등의 값을 이용하여 패킷 반입 여부를 결정한다. 즉, 사용 가능한 버퍼 크기를 초과하는 플로우에는 $strike_i$ 값을 증가 시켜 많은 값을 지니는 플로우에 대해 더 많은 패킷을 폐기하는 방법을 취한다. 그리고 각 플로우의 평균 큐 크기(avgcq)가 현재 라우터 버퍼에 활성화된 전체 플로우에(Nactive) 비례하여 전체 라우터의 평균 큐 크기(avg)를 넘지 않도록 하여 공정성을 증가 시킨다[2].

아래 표 3-10 ~ 3-13에서는 버클리 FRED에서 사용 되는 기본 파라메터 설명을 나타내며 표 3-11의 파라메터는 RED에서 사용되는 파라메터와 같다.

conn (p)	패킷 p의 연결 id
f(time)	time의 선형 함수

표 3-10. 매핑 함수 (FRED)

w_q	avg를 결정하기 위한 가중치
min_{th}	패킷의 폐기를 결정하는 최소 임계 값
max_{th}	패킷의 폐기를 결정하는 최대 임계 값
max_p	패킷 폐기를 위한 최대 확률값
min_q	플로우 당 최소 큐 크기

표 3-11. 상수 파라메터 (FRED)

Q	현재 큐 크기
Time	현재 시간
avg	평균 큐 크기
count	마지막 폐기된 패킷 이후의 패킷 수
avgcq	플로우 당 평균 큐 크기
Nactive	큐에 존재하는 플로우 수

표 3-12. 전역 파라메터 (FRED)

$qlen_i$	큐에 저장된 i 플로우의 패킷 수
$strike_i$	i 플로우에 대하여 fair share를 초과된 수

표 3-13. 플로우 당 파라메터 (FRED)

```
for each arriving packet p:
        if queue is empty
        //calculate average queue length
        if new flow(i)
            qlen_i ← 0, strike_i ← 0
        // identify and manage non-adaptive flows
          if((qlen_i ≥ max_q) || (avg ≥ min_th && qlen_i > 2 * avgcq)
                          || (qlen_i ≥ avgcq && strike_i > 1))
            strike_i + +
            drop packet p
            return
        if (min_th ≤ avg < max_th)
            // calculate probability p_a, with probability p_a :
            drop packet p
            return
        if Nactive
            avgcq ← avg / Nactive
        else
            avgcq ← avg
        avgcq ← MAX(avgcq, 1)
```

그림 3-8. FRED 알고리즘 의사 코드

(3) FRED 알고리즘의 문제점

FRED 알고리즘 역시 크게 두 가지의 문제점으로 나누어 생각해 볼 수 있다. 첫째, RED와 같이 고정된 파라메터 사용으로 인한 성능 저하 문제이고 둘째, 각 플로우마다 패킷 폐기를 결정하기 때문에 플로우 증가 시 복잡도가 증가라는 단점이 있다.

전자의 경우는 RED와 마찬가지로 Adaptive 개념을 도입하여 해결하였고, 후자의 경우는 TCP, UDP 플로우 따라 가상의 논리큐를 두는 방법으로 복잡도 문제를 해결 하였다[19][20]. 그러나 이전에 제안된 RED 알고리즘들은 또 다른 하나의 중요한 문제는 동일 링크에 다수의 TCP/UDP 플로우가 연결 되었을 때 패킷 폐기 확률 증가와 그에 따른 TCP 혼잡 제어에 의한 성능 저하 문제이다[2]. 이 문제 역시 TCP 프로토콜 레벨에서 접근한 연구가 있었고, RED 알고리즘에서 언급하고 있으나, 이와 관련된 연구가 많지 않고, 시뮬레이션 시나리오가 다소 한정적이고 미흡했던 부분이 있다. 즉, 그리 많지 않은 수(4~32개)의 플로우를 대상으로 시뮬레이션 하였거나, 토폴로지 구성을 TCP 플로우에만 국한했던 점이 바로 그것이다.

이 문제를 FRED에서는 “two packet buffer” 알고리즘 사용을 위한 many 옵션을 두어 다수의 플로우 연결 시 공정성이 깨지는 현상을 다소 감소 시켰다. 그러나 이 방법 역시 전송 프로토콜 레벨에서 다수의 흐름 제어를 하는 TCP 플로우와 하지 않는 UDP 플로우가 동시에 같은 링크를 공유할 때 TCP의 혼잡 제어로 인해 공평하게 자원을 공유할 수 없는 문제가 발생

한다. 다수의 TCP/UDP 플로우 기반에 FRED 알고리즘 사용 하여 발생하는 장점 및 단점을 확인하기 위해 관련 설명과 시뮬레이션을 통해 문제점을 제시한다.

a) 다수의 TCP 플로우의 특성 및 문제점

활성화된 모든 TCP 연결은 적어도 하나 이상의 패킷 전송을 유지하려 한다. 많은 연결들은 라우터 버퍼들을 가득 차게 유지 하려 하며 몇몇의 연결은 Time-out에 들어가게 된다. 라우터는 이 조건하에서 공정하게 버퍼들을 할당하여야만 한다. 또한 필요 없는 큐잉 지연 없이 플로우 수에 비례하여 버퍼 메모리를 추가 할 수 있어야 한다[2].

동일한 링크에 다수의 TCP 플로우가 연결 되었을 때, 해당 라우터에서 RED 혹은 FRED 같은 라우터 버퍼관리 알고리즘 사용 시 라우터 부하에 의해 평균 큐 크기가 종종 max_{th}을 초과해서 불공정한 Drop-Tail 라우터로 동작하도록 하는 원인이 된다.

이를 확인하기 위해 아래 그림 3-9과 같이 토폴로지를 구성하고 FRED 알고리즘을 사용해 시뮬레이션 하였다. 여기서는, FRED 라우터를 공유하는 호스트로 부터의 16개의 연결들은 60초 동안 실험 하였고, 이전과 동일한 파라메터값을 갖는다. (max_{th}= 8, min_{th}= 4, w_q = 0.002, max_p = 0.02, min_q = 2, q_{size} = 50)

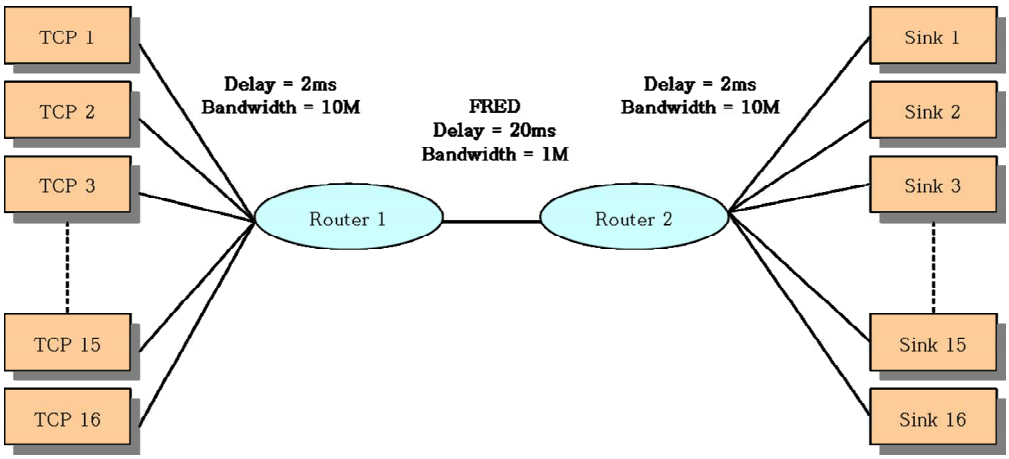

그림 3-9. 16개의 TCP 플로우 기반의 토폴로지

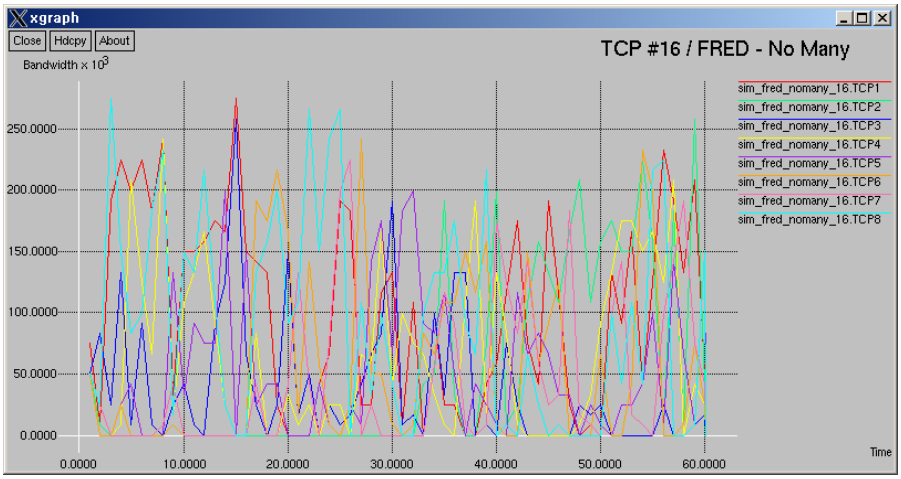

그림 3-10. 8개의 TCP 연결들의 대역폭 비교 (TCP #16/FRED)

그림 3-10는 임의의 8개 TCP 플로우의 대역폭 점유 추이를 보여 준다. 특정 순간에 출력 링크를 공유하는 연결들의 subset을 주목하라. 다른 연결들은 타임아웃 재전송에 의해 기다리고 있다. 이것은 단지 4 ~ 6개의 연결들에 min_q와 max_{th}를 할당 가능한 버퍼를 제공하는 것에 기인한다. 시뮬

레이션 결과와 같이 대역폭 분산이 심한 것을 알 수 있다. 이 문제는 타임아웃과 연관된 높은 대역폭 분산은 버퍼 메모리 추가로 다소 제거 할 수 있으나 높은 지연과 불공정한 버퍼 사용으로 인하여 바람직하지 못하다.

TCP는 매우 작은 혼잡 윈도우 크기로 인한 TCP 지수적 백오프 메커니즘 사용으로 높은 지연이 발생 될 수 있다. 이 문제를 D. Lin은 Nagle의 라운드 스케줄러 방식에 의한 "one packet switch" 을 확장하여 하나의 FIFO 큐 기반에서 “two packet buffer”라는 알고리즘을 내세워 이를 해결 하려 하였다[2]. 이 알고리즘은 FRED에서 각 flow 마다 가장 버퍼를 많이 점유하는 1~2개의 패킷을 유지하도록 하는 방법이며, 이 방법은 현재 FRED 알고리즘의 many 옵션을 통해서 동작 한다.

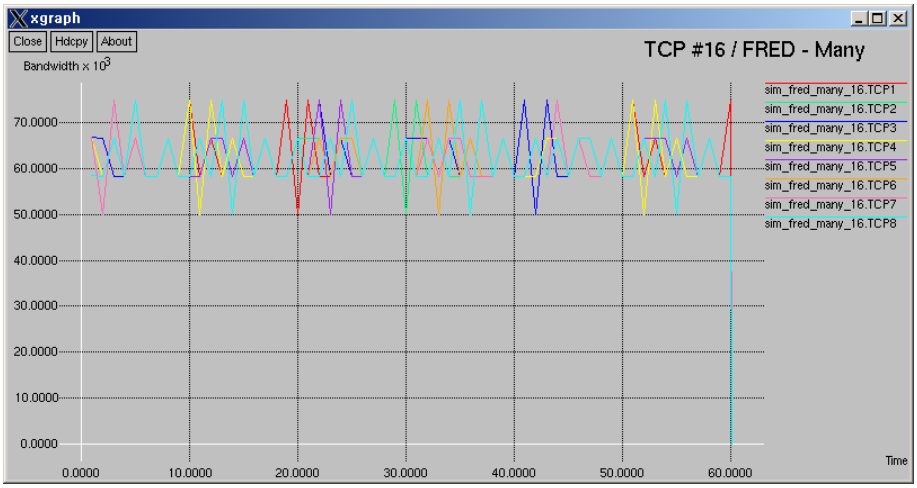

그림 3-11. 8개의 TCP 연결들의 대역폭 비교 (TCP #16/many)

many 옵션을 설정하지 않은 경우는 그림 3-10과 같은 결과를 나타내고, 설정하였을 경우는 그림 3-11과 같은 결과를 나타낸다. 즉, many 옵션을 설정하여 시뮬레이션 하였던 그림 3-11가 그렇지 않은 그림 3-10에 비해 공정성을 지표가 되는 대역폭 분산이 매우 낮아 진 것을 알 수 있다. 또한 표 3-14에서는 플로우 당 평균 패킷 폐기수, 평균 대역폭, 대역폭 분산 및 표준 편차를 나타내며, 패킷 폐기 수, 대역폭 점유 정도, 대역폭 분산값이 many 옵션을 사용 했을 경우 우수한 성능을 보인다. 표 3-14 또한 Mbps 단위로 계산된 대역폭의 분산값을 나타내었다.

옵 션	평균 폐기수	대역폭 점유율	대역폭 분산	대역폭 표준편차
미사용	95.125	97.15%	0.00033866	0.01840293
사용	72	98.81%	3.4880E-09	5.9060E-05

표 3-14. many 옵션 미사용 경우와 many 옵션 사용시 비교

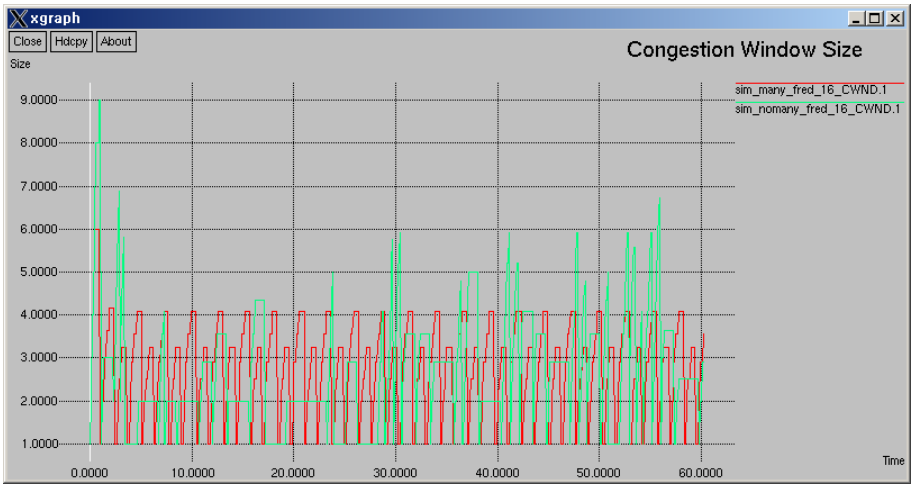

그림 3-12. many 옵션 미사용 시와 many 옵션 사용시 혼잡 윈도우

그림 3-12에서는 many 옵션 사용 시와 그렇지 않을 경우의 TCP 혼잡 윈도우 크기 변화를 보이며, 이를 통해서도 many 옵션 사용시가 그렇지 않을 경우 보다 안정적인 것을 확인 할 수 있다. 그러나 기존의 연구에 의하면 Fast Recovery 알고리즘이 적용 안된 TCP/Tahoe에 비해 작은 혼잡 윈도우에 대해 효과적으로 대처하는 TCP/Vegas 버전에서 보다 효과적으로 동작한다. 그러나 이 버전의 경우, 현재 많이 사용 하지 않으므로 TCP 레벨에서 혼잡 제어 메커니즘을 수정하여야 보다 좋은 성능을 발휘 한다는 문제가 있다[2].

```
//two packet mode (avg ≥ max_th)
  if  qlen_i ≥ 2
      count = 0;
      drop packet p;
      return;
```

그림 3-13. “two packet buffer” 의사 코드

“two packet buffer” 알고리즘은 many 옵션이 1로 설정 되어 있고, 평균버퍼 크기 avg가 최대 임계값 max_{th} 이상일 경우에 동작하며, 해당 플로우 큐에 저장된 패킷 수가 2를 넘는 경우 패킷을 폐기하는 알고리즘이다. 만약 many 옵션이 0으로 설정 되어 있을 경우에는 무조건 패킷 폐기를 하는 기본 FRED 알고리즘 대로 동작 한다.

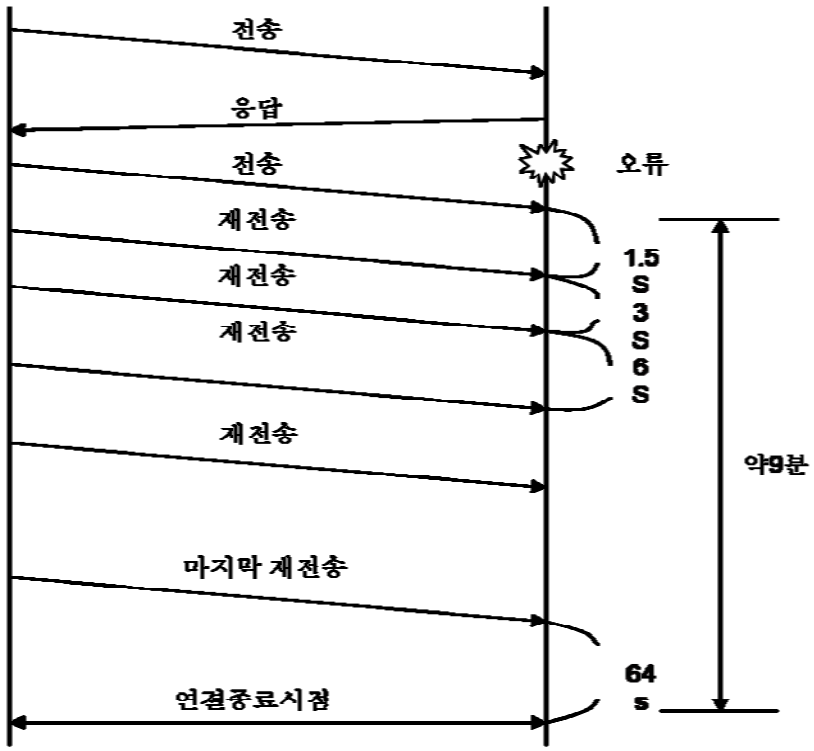

그림 3-14. TCP의 Exponential Backoff

그림 3-10, 3-11와 표 3-14에서 보는 바와 같이 다수의 플로우 연결 시 FRED 알고리즘에서 many 옵션을 설정하여 시뮬레이션 하면 대역폭 분산이 매우 줄어드는 것을 알 수 있다. many 옵션에서 구현된 그림 3-13의 “two packet buffer”은 기존의 FRED와는 달리 현재 평균 버퍼 크기가 max_{th}를 초과하는 경우 다수의 연결에 의한 혼잡 상황의 발생 시점이 가깝다고 인식하여 각 플로우 마다 $qlen_i$ 를 확인하여 2 보다 큰 경우에는 무조건 패킷을 폐기하는 방법으로 공정성을 유지 하며 그림 3-14에서는 TCP에서 높은 지연을 야기 시키는 원인이 되는 TCP의 Exponential-

Backoff 동작 과정을 보여 준다.

many옵션을 설정한 경우라도 다수의 TCP/UDP 플로우가 동시에 동일한 링크를 공유 하고, 현재 평균 버퍼 크기가 max_{th}를 초과하는 경우에는 불가피하게 패킷 손실이 발생하게 되고 TCP의 혼잡 제어 기법에 따라 타임아웃에 의한 Slow Start 단계로 자주 진입하게 되어 윈도우 크기가 줄어들어 불공정성이 발생하게 된다. many옵션을 사용 하였을 경우라도 다수의 TCP/UDP 플로우가 연결 되었을 때 공정성 파기 현상을 알아 보기 위해 두 가지 기준을 가지고 시뮬레이션을 위한 시나리오를 아래 표 3-15과 같이 구성 하였다.

시나리오 1	TCP flow No.8 vs TCP/UDP flow No.8
시나리오 2	TCP flow No.16 vs TCP/UDP flow No.16
시나리오 3	TCP flow No.32 vs TCP/UDP flow No.32

표 3-15. 다수의 TCP/UDP 기반의 공정성 확인을 위한 시나리오

라우터에서 일반적으로 4~6개의 연결들 정도까지만 min_q와 max_{th}를 할당 가능한 버퍼를 제공하는 것에 착안하여 공정성 파기 시점을 알아보기 위해 표 3-15는 최종적으로 총 32개의 플로우 연결까지 증가 시키면서 토폴로지를 구성하였다. 또한 시뮬레이션 시 사용되는 파라메터 값은 이전 실험과 동일한 조건으로 하였다. (max_{th}= 8, min_{th}= 4, w_q = 0.002, max_p = 0.02, min_q = 2, q_{size} = 50)

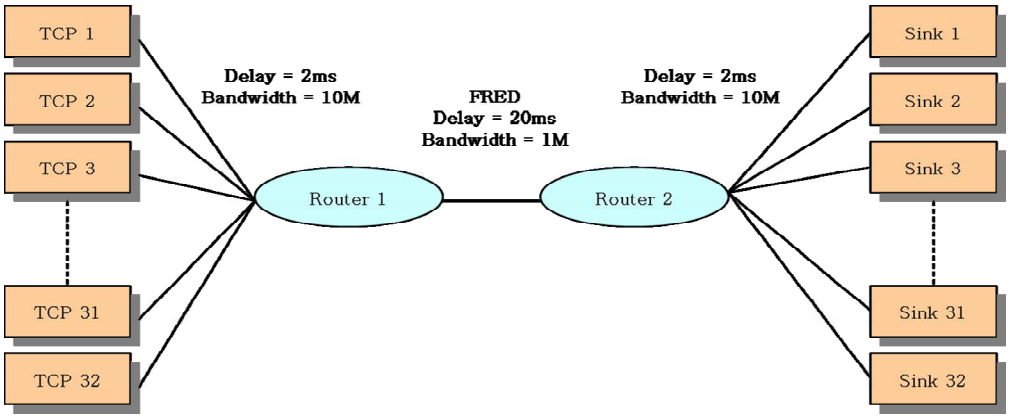

그림 3-15. TCP 플로우 기반의 토폴로지 (FRED/many)

그림 3-15은 TCP 플로우들의 연결로만 이루어진 경우를 나타내었고, 그림 3-16은 동시에 TCP/UDP 플로우들의 연결이 이루어진 경우를 나타내었다.

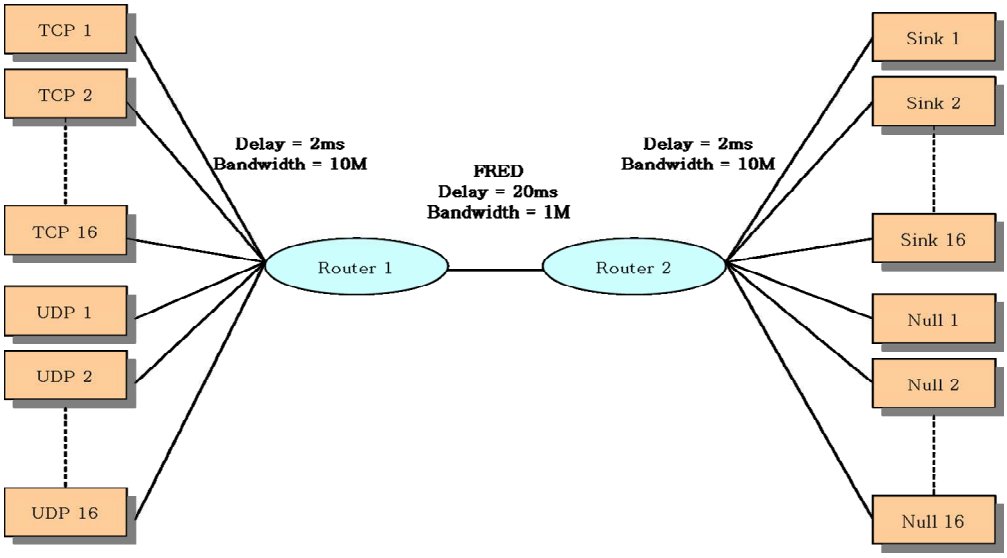

그림 3-16. TCP/UDP 플로우 기반의 토폴로지 (FRED/many)

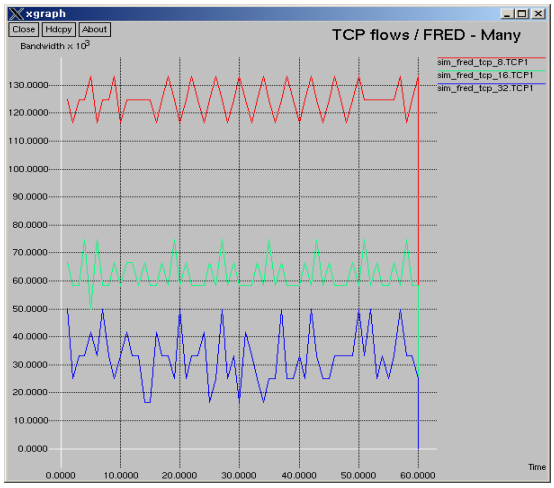

그림 3-17. TCP 플로우 기반의 첫 번째 TCP 플로우 대역폭 점유

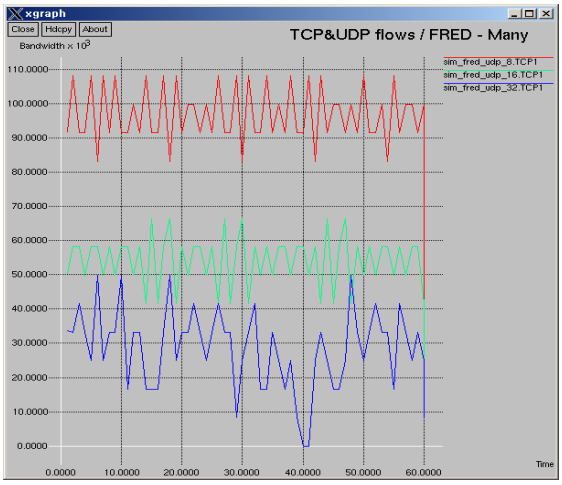

그림 3-18. TCP/UDP 플로우 기반의 첫 번째 TCP 플로우 대역폭 점유

그림 3-17, 3-18은 시뮬레이션 결과를 쉽게 알아 볼 수 있도록 플로우 수가 8, 16, 32로 증가 함에 따라 각 단계별 임의의 하나의 TCP 플로우의 대역 폭 점유 상태를 나타내었다. 그림 3-17은 그림 3-15의 토폴로지와 같이 TCP 플로우들로만 구성 되어 있고, 가장 상단에 위치하고 있는 선이

8개의 TCP 플로우 연결의 대역폭을 나타내고 있으며, 중간이 16개, 하단이 32개의 TCP 플로우 연결이 차지 하는 대역폭을 나타내고 있다. 마찬가지로 그림 3-18는 그림 3-16의 토폴로지와 같이 TCP/UDP 플로우가 링크를 공유하고 있는 구성이며, 선의 순서는 그림 3-17과 동일하다. 그림 3-17, 3-18을 통해 알 수 있는 부분은 연결된 플로우 수가 많을수록 각 플로우가 차지하는 대역폭이 줄어 들고, 대역폭 분산이 커지는 것을 알 수 있다. 또한 TCP연결로만 이루어진 구성 보다는 TCP/UDP 플로우가 혼합되어 이루어진 구성의 경우가 각 플로우 마다 대역폭 분산이 좀 더 큰 것을 알 수 있다. 즉, 가장 주목할 만한 점은 TCP/UDP 플로우 개수가 32개로 증가 하면서 급격히 점유 대역폭 분산이 커지는 현상을 볼 수 있다.

아래 표 3-16에서는 위 시뮬레이션의 결과를 토대로 공정성의 지표가 되는 대역폭 표준편차와 혼잡 윈도우 크기의 표준 편차를 나타낸다. (8개의 플로우로 전체 대역폭 추정)

	Flow #8		Flow #16		Flow #32	
	TCP	TCP/UDP	TCP	TCP/UDP	TCP	TCP/UDP
B	97.37%	98.46%	97.81%	98.69%	98.78%	99.01%
BV	9.3E-09	1.3E-07	3.4E-09	1.6E-06	1.2E-06	0.0001
QA	19.53	19.61	19.25	19.75	19.29	19.77
QV	63.80	62.41	67.78	60.4	68.66	61.97

표 3-16. 평균 대역폭 점유율, 분산 및 평균 큐 점유 크기, 분산

아래 그림 3-19, 3-20은 플로우 개수가 가장 큰 32개 일 때 TCP, TCP/UDP 기반의 TCP 플로우의 대역폭 점유도 변화 그래프로써 플로우 개수가 16개를 초과하여 32개에 이르러서는 급격히 분산이 높아 지는 것을 알 수 있다.

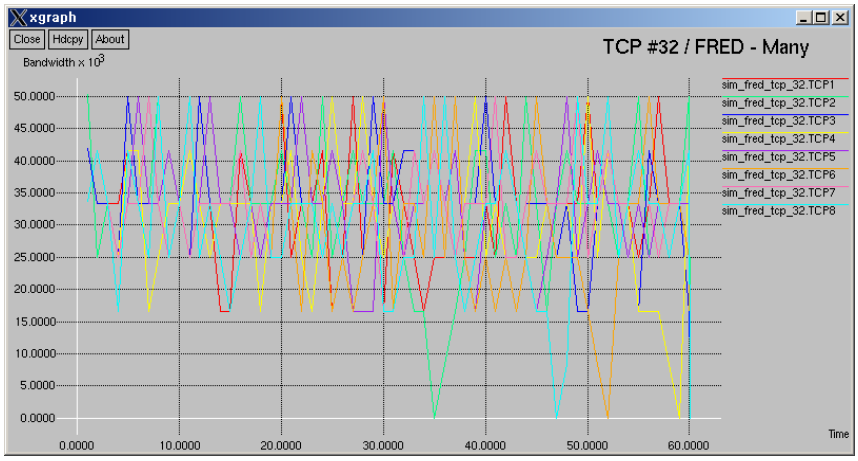

그림 3-19. 32개의 TCP 플로우 연결들의 대역폭 점유 (many)

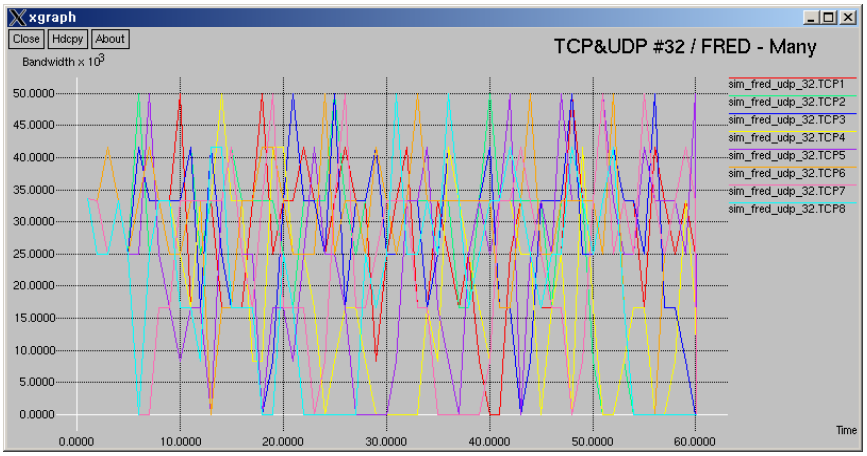

그림 3-20. 32개의 TCP/UDP 플로우 연결들의 대역폭 점유 (many)

위의 시뮬레이션 결과를 통해 FRED 알고리즘에서 다수의 플로우를 지원하는 “two packet buffer” 알고리즘으로 동작하게 하는 many 옵션을 설정하더라도 특정 수 이상의 많은 TCP/UDP 플로우가 동시에 연결함에 따라 공정성이 깨지게 되는 현상을 살펴 볼 수 있었다.

제 4 장 실험 결과 및 분석

본 장에서는 3장에서 언급 하였던 TCP 버전에 따라 다수의 TCP/UDP 플로우 환경에서 FRED의 “two packet buffer” 알고리즘을 이용하여 시뮬레이션을 통하여 발생 되는 문제점을 알아보고, 제시한 “specific random drop” 알고리즘을 적용한 Many-FRED를 실험적으로 적용하여 시뮬레이션 후 기존의 방법과 비교, 분석 한다.

제 1 절 실험 환경

시뮬레이션은 unix 환경에서 NS-2.28을 사용하였고, 크게 두 가지 경우로 나누어 수행하였다. 첫째, 각 TCP 버전 별로 플로우를 32, 64, 128개로 증가 시켜 가며 상태를 비교 하였으며, 둘째, Fast Recovery 알고리즘이 포함 되어 있지 않는 TCP/Tahoe 기반에서 “two packet buffer” 알고리즘을 확장한 “specific random drop”를 실험적으로 적용하여 시뮬레이션 후 앞선 TCP/Tahoe 결과와 비교 분석 하였다. 두 경우 모두 30초 동안 실험하였고, TCP 전송측 에서는 FTP를 이용하였으며, UDP 전송측에서는 CBR 데이터를 이용 했다. TCP 버전은 TCP/Tahoe, TCP/Newreno, TCP/Vegas를 대상으로 하였으며, 그림 4-1과 같이 토폴로지를 구성 하였고, 시뮬레이션 시 사용되는 파라메터 값은 이전 실험과 동일한 조건으로 하였다. (max_{th}= 8, min_{th}= 4, w_q = 0.002, max_p = 0.02, min_q = 2, q_{size} = 50).

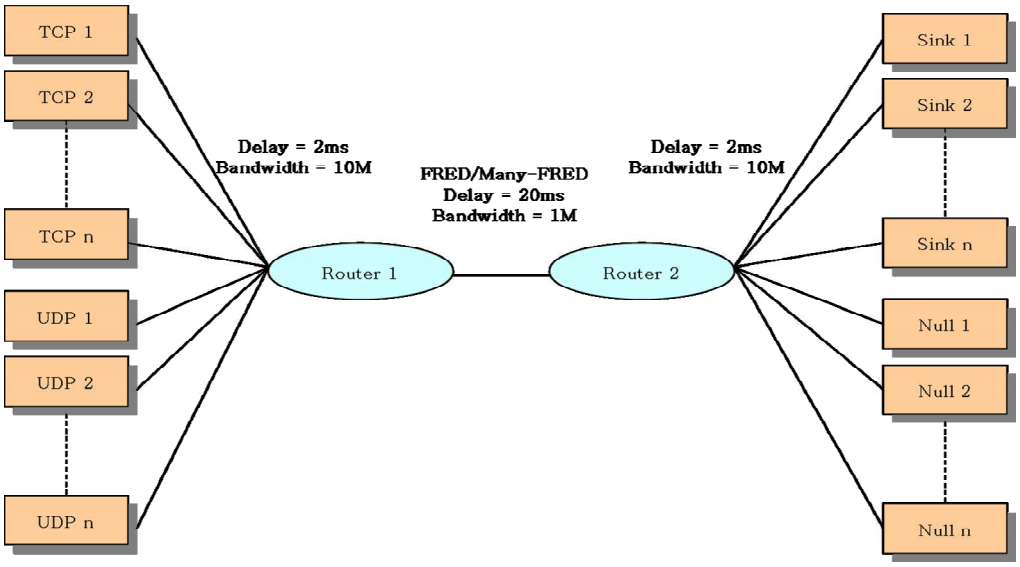

그림 4-1. 다수의 TCP/UDP 플로우 기반의 토폴로지

위의 토폴로지와 같이 혼잡 구간은 그렇지 않은 곳의 1/10의 대역폭을 할당하여 극심한 혼잡 상황을 구현 하였다. 그리고 아래와 같이 플로우 개수별로 시나리오를 구성하였다.

시나리오 1	16개의 TCP 플로우 + 16개의 UDP 플로우(총 32개)
시나리오 2	32개의 TCP 플로우 + 32개의 UDP 플로우(총 64개)
시나리오 3	64개의 TCP 플로우 + 64개의 UDP 플로우(총 128개)

표 4-1. 시뮬레이션을 위한 시나리오 구성

각 시나리오에서는 버클리 대학의 기본 FRED 알고리즘을 사용하였고, TCP 트래픽과 UDP 트래픽이 동일한 조건하에서 시뮬레이션이 가능하도록

데이터 발생 시간은 TCP, UDP 모두 동일한 조건으로 하였다. 그리고 FRED 알고리즘에 포함되어 있는 many 옵션을 1로 설정하여 “two packet buffer” 알고리즘을 사용하였다.

파라메터	설 정 값
w_q	0.002
max_p	0.02
max_{th}	8
min_{th}	4
min_q	2
q_{size}	50

표 4-2. 시뮬레이션을 위한 FRED, Many-FRED 파라메터 값

앞서 살펴 본 바와 같이 FRED 알고리즘 기반에서 다수의 TCP/UDP 플로우 연결 시 발생하는 max_{th}가 avg(평균 버퍼 크기)를 초과해서 발생되는 공정성 문제를 해결하기 위해 본 논문에서는 이런 한계점을 바탕으로 “two packet buffer”을 변형하여 기존 FRED을 확장 하였다. 또한, 기존의 일반적인 연구에서의 시뮬레이션은 FRED 라우터 환경에서는 보통 4~32개 정도 였으나 일반적인 라우터에서 제한되는 큐 크기와 앞의 시뮬레이션 결과에 따라 공정성 급격히 파기 되는 32개 이상의 플로우에 대해서도 적용 가능 하도록 하였다. 시뮬레이션은 먼저 TCP 버전 별로 FRED 알고리즘의 many 옵션을 설정하여 수행하며, 그 후 아래에 제시한 Many-FRED를

TCP/Tahoe에 적용해 그 결과를 토대로 기존의 FRED와 성능을 비교 분석 한다.

a) Many-FRED

본 논문에서 확장하는 FRED알고리즘은 다수의 TCP/UDP 플로우 연결 시 공정성 유지를 목적으로 하여 앞으로 Many-FRED라 부르기로 한다. 해당 라우터에 반입(enqueue)되는 패킷에 대해 avg가 max_{th}를 초과 할 경우 다수의 플로우들이 경쟁하고 있는 상태로 인식한다. 실질적으로 문제가 되는 트래픽은 TCP이며, 이는 TCP 혼잡 제어에 의한 타임아웃으로 인한 성능 저하이다. 그러나 이 문제를 달리 생각해 보면 TCP가 혼잡 제어를 하기 때문이 아니라 UDP가 혼잡 제어를 하지 않기 때문이라 볼 수 있다. 즉, TCP 플로우의 경우 혼잡 상황 시 불가피하게 패킷 폐기로 인한 타임아웃에 걸리게 되고 이 틈을 타서 전송률 제어 하지 않는 UDP 플로우가 TCP 대역폭을 점유하기 때문이다. 이 문제를 해결하기 위해서 전송 프로토콜 레벨에서는 앞서 2장에서 언급한 TFRC를 통해 보완하였다. 그러나 TFRC의 경우 알고리즘 특징으로 일반적으로 RTP/RTCP 기반에서 사용된다. 만약 단순히 UDP 기반에서 사용하려면 응용 계층에서 TFRC에 필요한 정보를 계산 하는 부분이 추가 되어야 한다. 또한 TFRC 역시 알고리즘 특성으로 인해 다수의 플로우 연결 시에는 문제가 발생하며, FRED와 마찬가지로 이를 극복하기 위해 연구가 이루어 지고 있다. 그리고 흐름 제어를 하지 않는 트래픽을 전송 하는 모든 전송측이 TFRC를 이용하여 데이터를 전송

한다는 보장은 없다. 이러한 이유로 본 논문에서는 FRED 라우터 기반에서 혼잡 감지 시 흐름 제어를 하지 않는 UDP 트래픽에 대해 패널티를 좀 더 부여하여 기존 FRED 보다 UDP 패킷을 많이 폐기 하도록 하여 UDP 대역폭을 TCP에게 나누어 주어 공정성 향상에 도움이 되려고 한다.

Many-FRED 알고리즘은 “two packet buffer” 알고리즘을 토대로 확장 하였으며, 현재 라우터에 반입 되는 패킷에 대해 헤더 정보를 확인하고, 만약 흐름 제어를 하는 TCP 트래픽인 경우는 기존의 “two packet buffer” 단계로 들어가도록 하고, 흐름 제어를 하지 않는 UDP 트래픽인 경우에는 현재 라우터의 평균 버퍼 크기가 버퍼 임계치 이상으로 증가 하고 “two packet buffer” 알고리즘에 따라 해당 플로우의 버퍼에 남은 패킷이 2개 이상일 경우에는 “specific random drop” 알고리즘이 동작 하게 된다. “specific random drop” 알고리즘에서는 현재 각 플로우의 평균 버퍼 크기에 따라 최소 임계치에서 최대 임계치 사이의 범위에서는 버퍼에 반입되는 UDP 트래픽은 RED 알고리즘의 확률 계산식에 따라 폐기하며, 최대 임계치를 넘어서면 확률값을 1로 만들어 흐름제어를 하지 않는 트래픽이 버퍼에 반입되는 것을 다소 줄어들게 한다. 이로 인해 흐름제어를 하는 TCP의 플로우에게 UDP에서 빼앗은 가용 대역폭 나누어 주어 공정성을 보장해 주는 알고리즘이다.

```
//enqueue
if pkt_header = udp
    pkt_udp ← true
 else
    pkt_udp ← false
if avg ≥ min_th &&   avg < max_th
    if pkt_udp
        // with probability p_a :
        drop packet p
        count ← 0
        return;
else if avg ≥ max_th
    if len_i ≥ 2 && pkt_udp
        count ← 0
        drop packet p
        return
```

그림 4-2. Many-FRED 를 위한 “specific random drop” 의사코드

제 2 절 실험 결과

	Flow #32			
	대역폭 점유율	플로우간 분산	큐 평균 점유	큐 분산
Tahoe	99.10%	0.050742338	24.96716	0.533117
Newreno	96.76%	0.044736344	24.9644	0.607991
Vegas	99.00%	0.030787	24.97066	0.514689

표 4-3. TCP 버전 별 32개 플로우 연결에 의한 비교

	Flow #64			
	대역폭 점유율	플로우간 분산	큐 평균 점유	큐 분산
Tahoe	99.36%	0.229418876	24.98674	0.225233
Newreno	98.89%	0.148193355	24.98639	0.231151
Vegas	99.21%	0.101677	24.98593	0.238918

표 4-4. TCP 버전 별 64개 플로우 연결에 의한 비교

	Flow #128			
	대역폭 점유율	플로우간 분산	큐 평균 점유	큐 분산
Tahoe	99.27%	0.228153391	24.98621	0.234317
Newreno	98.78%	0.194496832	24.9863	0.232787
Vegas	99.37%	0.242794	24.98745	0.213188

표 4-5. TCP 버전 별 128개 플로우 연결에 의한 비교

표 4-3, 4-4, 4-5는 시뮬레이션 결과를 정리 한 것이며, 각 경우마다 임의의 8개의 TCP 플로우와 8개의 UDP 플로우를 선택하여 측정한 결과이다. 플로우간 분산은 Mbps로 계산하여 나타내었다. 시뮬레이션 결과 Tahoe의 경우 플로우가 64개로 증가 하였을 경우 급격히 TCP 대역폭이 감소 하고 그 여분을 UDP 트래픽이 차지 하였으며, 64개의 플로우 경우나 128개로 증가하였을 경우가 서로 비슷한 대역폭 점유 수치를 보였다. 또한 32개 플로우 연결 시에는 그림 4-4와 같이 종종 타임아웃에 의한 패킷 전송을 하지 못하는 플로우들이 있는 것을 알 수 있다. 반면 Newreno와 Vegas의 경우는 64개의 플로우 연결 시에 TCP/Tahoe에 비해 보다 TCP 대역폭이 증가 하였음을 알 수 있었고 32개의 플로우 연결 시 TCP/Vegas가 가장 Slow Start 단계로 적게 진입 하는 것을 알 수 있다. 그러나 두 가지 버전 역시 128개까지 플로우가 증가 하면 TCP 플로우는 거의 대역폭을 차지 하지 못하는 현상을 나타낸다.

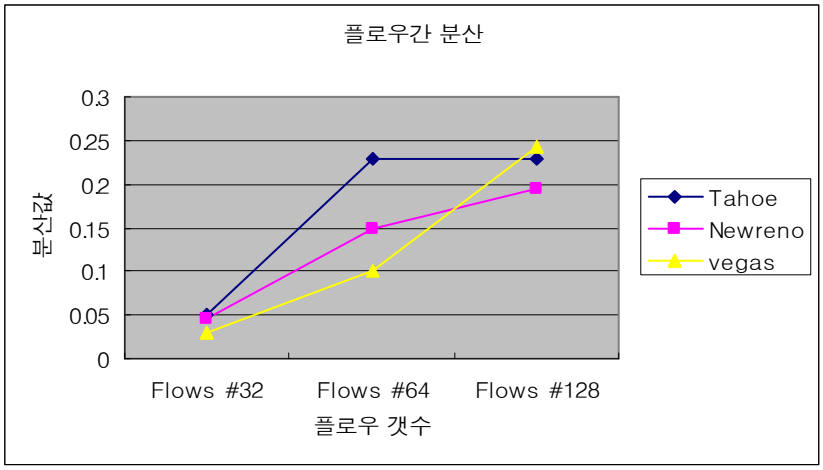

그림 4-3. 다수 플로우 기반의 TCP 와 UDP 플로우 사이의 대역폭 분산

그림 4-3 는 각 TCP 버전 별 TCP 트래픽과 UDP 트래픽간의 대역폭 분산 정도를 나타낸 것이며, TCP/Tahoe 의 경우 플로우가 64 개로 증가하면서 대역폭 분산 정도가 매우 커지는 것을 알 수 있다. 또한 Vegas 의 경우 플로우 64 개인 일 때 다른 버전에 비해 가장 낮은 대역폭 분산을 가지는 것을 확인 할 수 있다. 그러나 플로우가 개수가 128 개로 증가함에 따라 TCP 플로우의 대역폭 점유율이 급격히 감소하며 대부분의 대역폭을 UDP 가 차지 하여 세 가지 TCP 버전에 거의 비슷하게 높은 서로 다른 종류의 플로우간의 대역폭 분산이 매우 높아 진다.

그림 4-4 에서 4-12까지는 TCP 버전에 따른 임의의 8개의 TCP 플로우들의 대역폭 점유 상태를 그래프로 나타내었다. 그림 4-3에서 나타낸 분산, 점유 정도를 그래프를 통해 보다 쉽게 확인 할 수 있다. 8개의 TCP 플로우만을 대상으로 하여 정확도가 떨어 질 수 있으나 TCP 플로우에 대해 한눈에 판단하기 쉽도록 나타내었다.

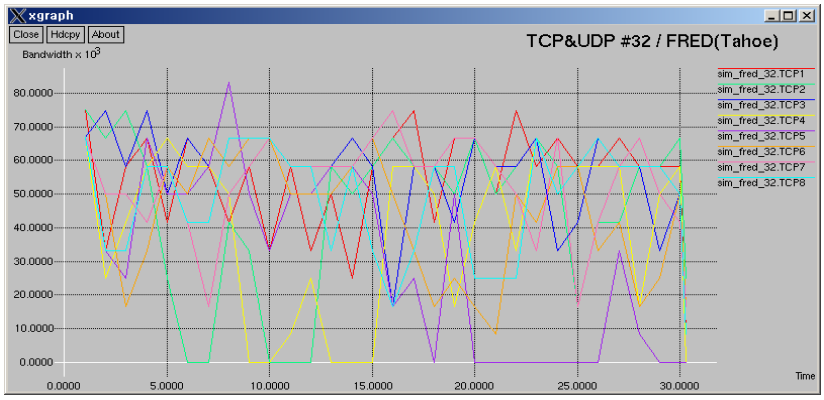

그림 4-4. TCP 플로우의 대역폭 변화 (Tahoe/Flows #32)

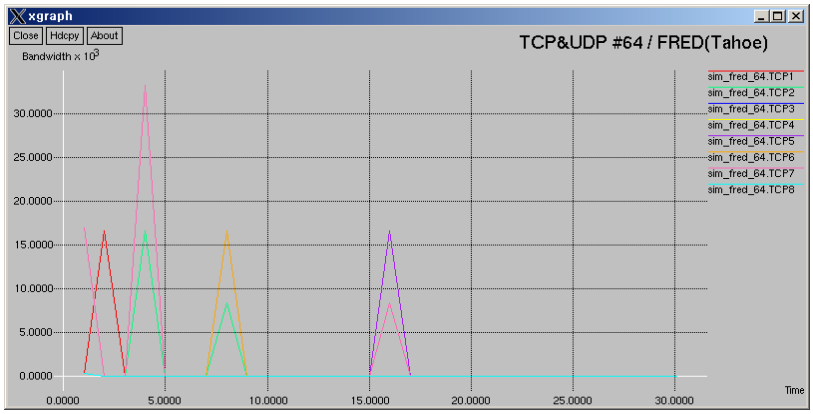

그림 4-5. TCP 플로우의 대역폭 변화 (Tahoe/Flows #64)

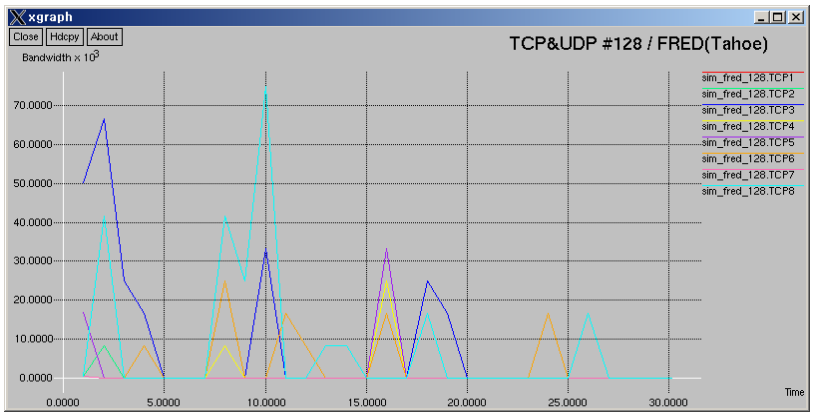

그림 4-6. TCP 플로우의 대역폭 변화 (Tahoe/Flows #128)

그림 4-4 의 그래프와 같이 플로우 개수가 32 경우 TCP/Tahoe 기반에서는 TCP 플로우가 혼잡 상황으로 인한 타임 아웃에 걸려 대역폭을 점유 못하는 플로우를 종종 볼 수 있으며, TCP 플로우별 대역폭 분산 정도도 큰 것을 알 수 있다. 그리고 플로우 개수가 64 개 경우와 128 개 일 경우 모두 TCP 대역폭 점유율이 급격히 감소 한 것을 알 수 있고, 플로우 개수가

64 개 일 경우나 128 개 일 경우가 TCP 대역폭 점유가 비슷하다는 것을 위의 표의 결과 값을 통해서 알 수 있다.

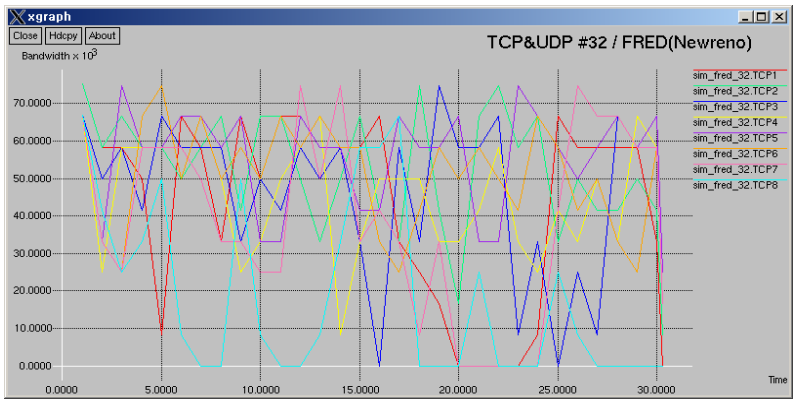

그림 4-7. TCP 플로우의 대역폭 변화 (Newreno/Flows #32)

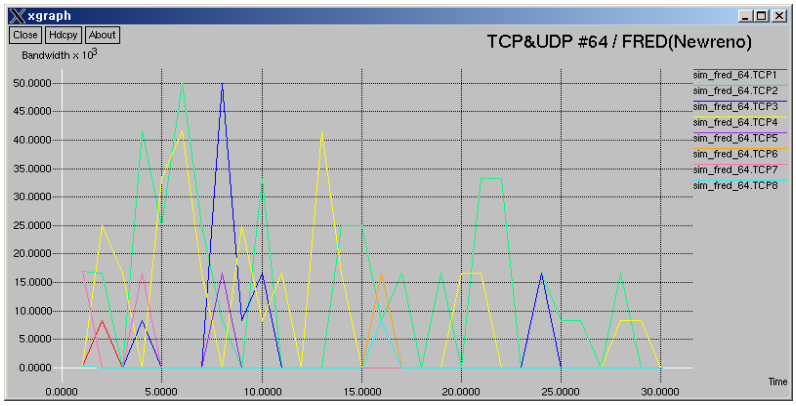

그림 4-8. TCP 플로우의 대역폭 변화 (Newreno/Flows #64)

TCP/Newreno의 경우 플로우 개수가 32개일 경우는 TCP/Tahoe와 큰 차이가 없으나 64개 일 경우는 TCP/Tahoe에 비해 TCP/Tahoe에 비해 대역폭 점유율이 좀 더 높은 것을 알 수 있다.

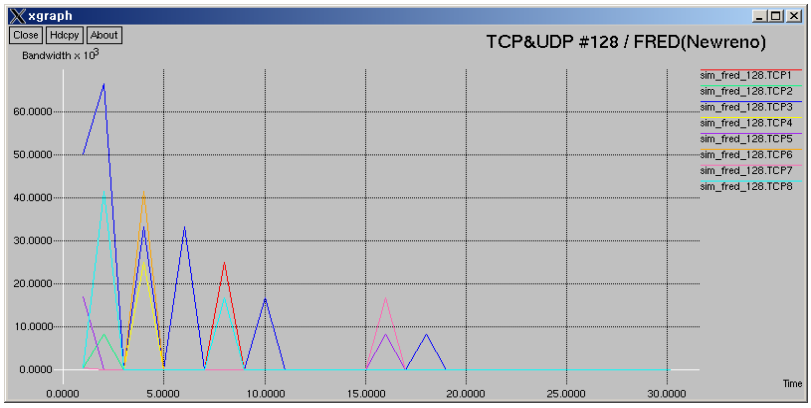

그림 4-9. TCP 플로우의 대역폭 변화 (Newreno/Flows #128)

위의 시뮬레이션에서 가장 특징적인 점은 플로우 개수가 32 개일 경우 TCP/Vegas 결과이다. RTT 기반으로 작은 혼잡 윈도우 크기에서도 효과적으로 대응하는 혼잡 제어기법으로 인해 다른 두 가지 버전에 비해 타임 아웃에 의한 성능 저하가 눈에 띄게 줄어 든 것을 알 수 있다.

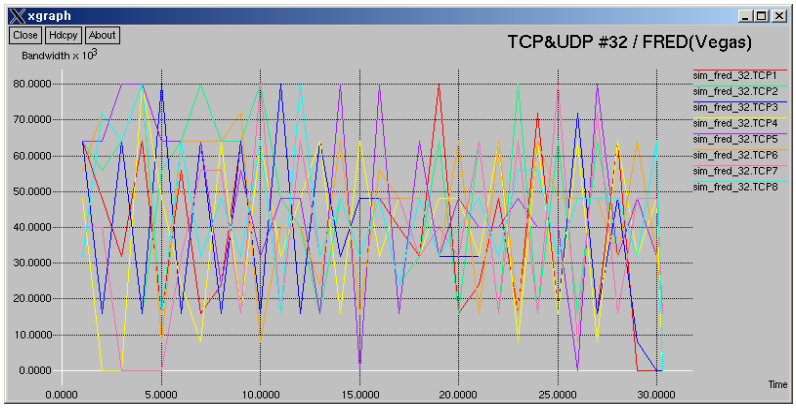

그림 4-10. TCP 플로우의 대역폭 변화 (Vegas/Flows #32)

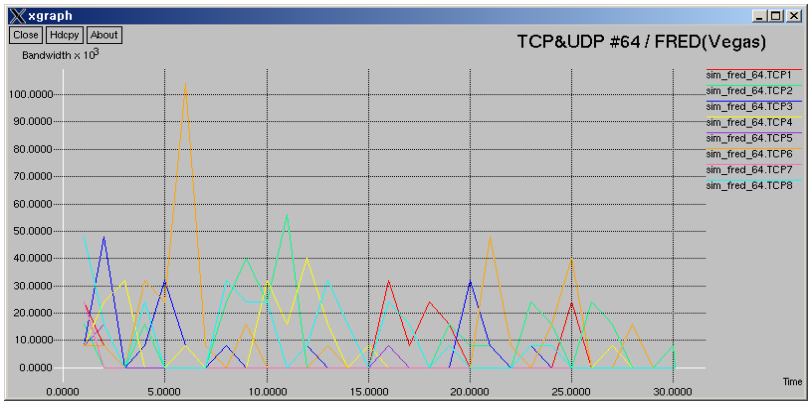

그림 4-11. TCP 플로우의 대역폭 변화 (Vegas/Flows #64)

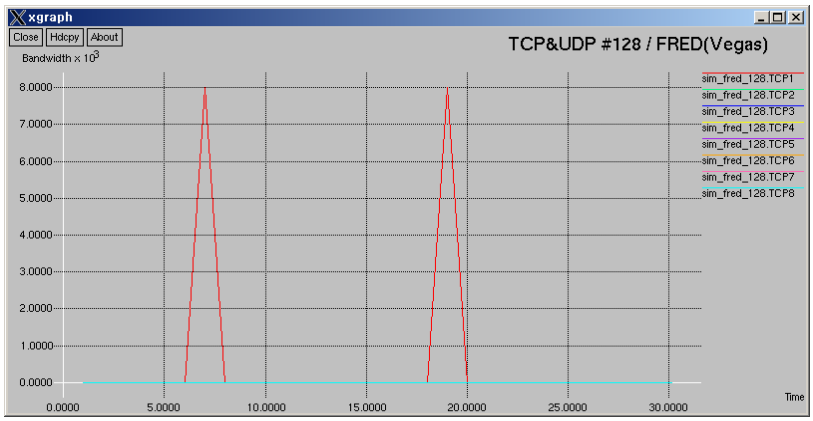

그림 4-12. TCP 플로우의 대역폭 변화 (Vegas/Flows #128)

TCP/Vegas 의 경우 그림 4-11 과 같이 플로우 개수가 64 개 까지 증가해서도 TCP/Tahoe 에 비해 높은 대역폭 점유율을 보이며, 세 가지 TCP 버전 중에서 가장 대역폭 분산이 낮아 좋은 방법이라는 것을 알 수 있다. 그러나 대역폭 분산 비교 시 TCP/Tahoe 의 경우는 TCP 플로우가 타임아웃에 걸려 패킷을 전송 못해서 대역폭 점유율이 0%인 경우가 많으므로 결과 비교 시 그 부분을 고려해야 한다.

그러나 TCP/Vegas 의 경우 플로우 개수가 128 개로 늘어나면 다른 두 가지 TCP 버전에 비해 대역폭 점유율 가장 낮아 지는 것을 알 수 있다.

이번에는 앞의 실험 결과에 따라 가장 성능이 안 좋은 TCP/Tahoe 환경에서 성능을 향상 시키기 위하여 제안한 Many-FRED를 통해 시뮬레이션을 하여 기존의 결과와 비교 하도록 한다. 모든 시뮬레이션 토폴로지 구성과 파라메터 값은 이전 FRED 기반의 TCP/Tahoe와 같으므로, 아래 표 4-6과 그림 4-13 ~ 4-16의 결과를 참조한다.

	Many-FRED (Tahoe)			
	대역폭 점유율	플로우간 분산	큐 평균 점유율	큐 분산
Flow #32	96.41%	0.046166	24.95993	0.635062
Flow #64	98.67%	0.096572	24.96705	0.531011
Flow #128	98.92%	0.140433	24.95181	0.867014

표 4-6. Many-FRED 적용 시 연결 개수에 의한 비교

그림 4-14, 4-15, 4-16를 통해 기존의 TCP/Tahoe에서 FRED을 적용 하였을 경우와 비교해 보면 플로우 개수가 64일 경우와 128경우 기존의 방법 보단 TCP가 보다 많은 대역폭을 차지 하고 있는 것을 알 수 있다.

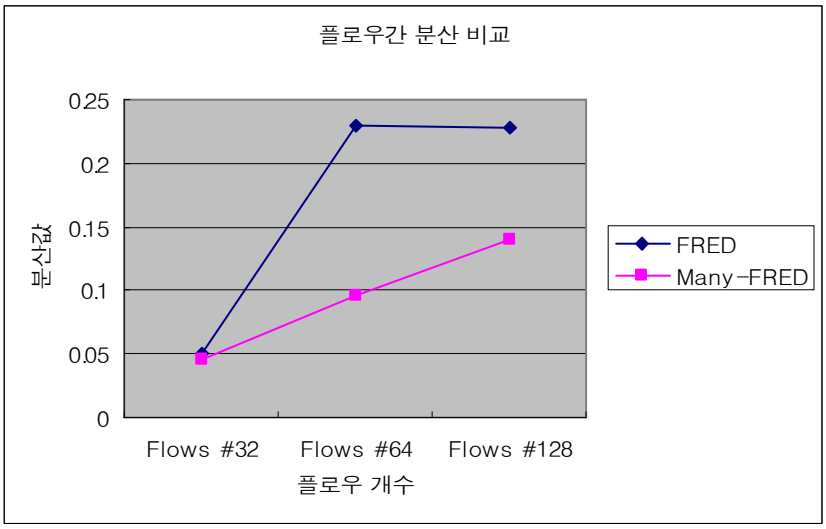

그림 4-13. FRED 와 Many-FRED 의 TCP/UDP 플로우간 분산 비교

그림 4-13은 many 옵션을 적용한 FRED와 이를 수정하여 확장한 Many-FRED에 대해 TCP 플로우와 UDP 플로우간의 분산 정도를 비교 하였다. 이 경우 역시 8개 의 TCP 플로우를 대상으로 결과를 산출 하였으므로 정확도가 다소 떨어 질 수 있으나 플로우 개수가 64개에서 128개 까지 증가하는 동안 기존의 방법 보다는 분산이 평균적으로 약 0.1 정도 낮아 진 것을 시뮬레이션 결과를 통해 확인 할 수 있다. 그러나 이 방법 역시 이전에 시뮬레이션에서 TCP/Vegas 버전에 플로우 32개 일 경우 상태와 비교하면 타임아웃에 자주 걸리게 되는 것을 알 수 있다. 아래 그림 4-14, 4-15, 4-16은 Many-FRED 기반에서 앞의 시뮬레이션과 같이 처음 8개의 TCP 플로우들의 대역폭 점유 변화 추이를 보여 준다.

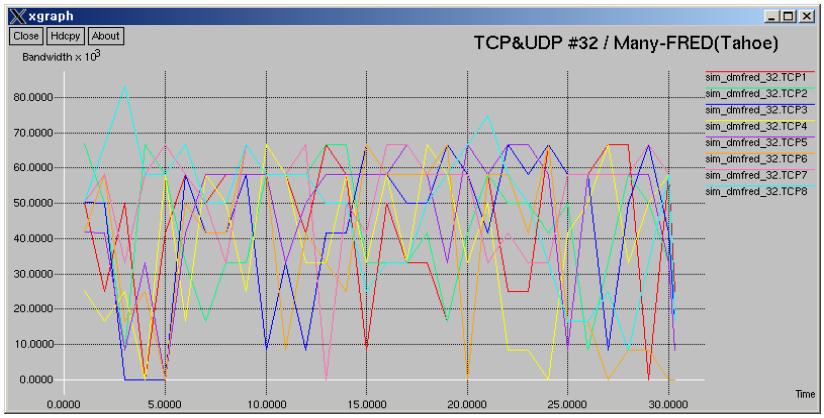

그림 4-14. TCP 플로우의 대역폭 변화 (Many-FRED/Flows #32)

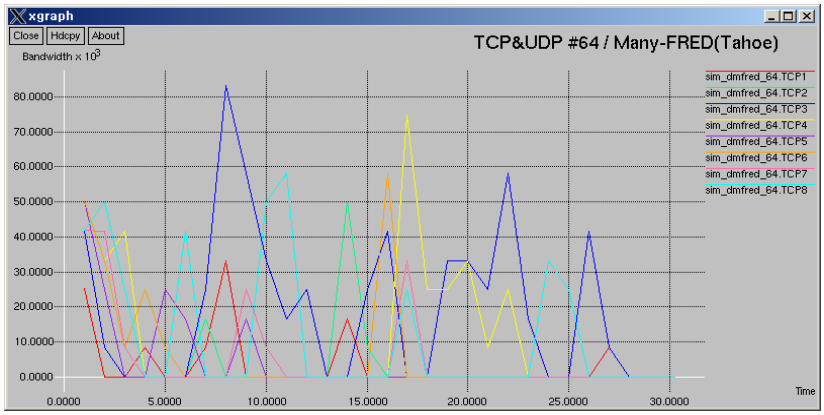

그림 4-15. TCP 플로우의 대역폭 변화 (Many-FRED/Flows #64)

TCP/Tahoe 에서 Many-FRED 를 적용한 경우 플로우 개수가 32 개 일 때는 FRED 의 경우에 비해 TCP 패킷 폐기에 의한 타임 아웃이 줄어 든 것을 알 수 있다. 또한 플로우 개수가 64 개, 128 개로 증가 하였을 경우에 FRED 에 비해 TCP 플로우의 대역폭 점유율이 증가 했음을 알 수 있다. 그러나 TCP 플로우 대역폭 분산이 커진 것을 확인 할 수 있다.

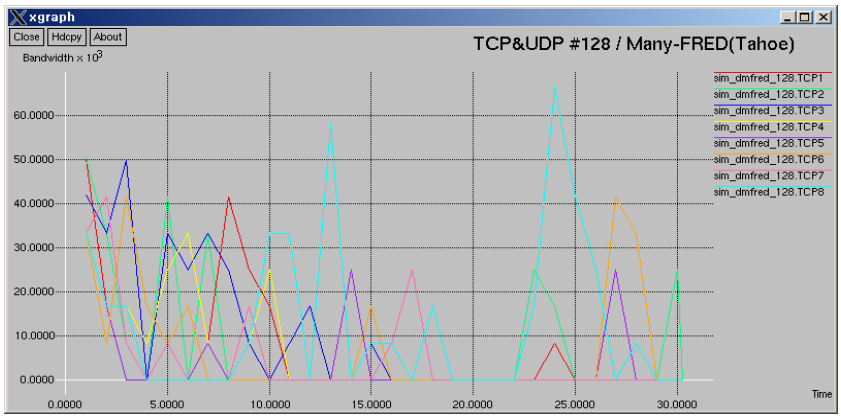

그림 4-16. TCP 플로우의 대역폭 변화 (Many-FRED/Flows #128)

제 3 절 실험 분석

지금까지의 시뮬레이션을 살펴 보면 기존의 FRED 알고리즘에서의 many 옵션을 설정하여 TCP 버전 별로 플로우 개수를 증가 시키면서 시뮬레이션 한 결과, TCP/Tahoe의 경우 64개의 플로우 연결 시에도 TCP 대역폭 점유율은 128개의 연결과 비슷한 정도로 0%에 가깝게 매우 낮아 졌다. 그리고 TCP/Newreno와 TCP/Vegas의 경우에는 64개의 플로우 연결 까지는 TCP/Tahoe에 비해 더 많은 대역폭을 점유 하나, 128개의 플로우 연결에 이르러서는 TCP/Tahoe와 마찬가지로 TCP 대역폭 점유율이 매우 낮게 나타났다. 이러한 문제의 주된 이유는 흐름제어를 하지 않는 UDP 트래픽에 의한 것이므로 Many-FRED 알고리즘을 사용하여 다수의 플로우 연결 시에 UDP 트래픽 반입을 줄이도록 UDP 패킷을 좀 더 폐기하는 방법을 제시하여 실험적으로 적용해 보았다. 그 결과 그림 4-13에서와 같이 TCP/Tahoe에서도 플로우간 대역폭 분산 이 약 0.1 정도 낮아 졌으며,

TCP/Tahoe 사용 시 기존의 FRED에 비해 TCP 대역폭 점유율이 대략 5~10% 정도 증가 했다. 그리고 모두 TCP 버전에서 동일하게 64개 이상의 플로우가 연결된 상태에서는 큐 분산이 매우 낮아져 버퍼를 효율성 있게 사용 하고 있는 것을 확인 할 수 있었다. 그러나 이 시뮬레이션의 결과 산출은 임의의 8개의 TCP 플로우에 대한 국한 된 것이므로 정확도가 높지 않고 본 논문에서 시뮬레이션 시 구성 하였던 토폴로지 보다 더욱 빠른 속도로 UDP 트래픽을 전송하거나 더욱 극심한 혼잡 상황을 구현하면 그 결과가 매우 다르게 나타날 수 있을 것이다. 그리고 시뮬레이션에서 64개의 이상으로 플로우 수가 증가 하였을 경우는 TCP/Newreno 와 TCP/Vegas 기반인 경우 TCP/Tahoe 보다는 TCP 대역폭이 증가하나 TCP 플로우 별 분산이 매우 커지는 것을 알 수 있었으며, Many-FRED 경우 역시 TCP 플로우 별 분산이 매우 커져서 바람직하지 못하다. 이러한 현상은 TCP 혼잡 제어에 특징에 의한 것이므로 단순히 FRED 알고리즘 변형만으로는 다소 무리가 있다.

본 논문에서 FRED을 확장한 Many-FRED를 통해 실험적으로 시뮬레이션 한 결과 토대로 살펴보면 다수의 플로우 경우 보다 UDP 패킷을 많이 폐기 시켜서 그 결과로 TCP 플로우에 대해 대역폭을 약간 증가 시켜 줄수는 있을지 모르나 근본적인 해결책은 되지 못하는 것으로 분석 된다. 그 이유는 단순히 TCP 대역폭 증가에 의한 서로 다른 종류의 플로우간의 분산 정도가 낮아진 점을 빼고는 TCP 플로우별 분산이 매우 크고 타임아웃에 의한 Slow Start로 많이 진입하기 때문이다. 그러한 이유로 인해 FRED 알고리

즘은 다수의 서로 다른 종류의 플로우가 동시에 같은 링크를 공유 하였을 경우 성능이 급격히 저하 되므로 바람직하지 못한 알고리즘이다. 그러나 다소 경미한 혼잡 상황 또는 동시에 너무 많지 않는 수의 (대략 32개 미만)플로우가 연결 되었을 경우 FRED를 사용하여서도 파라메터를 동적으로 변화 시키거나 큐 크기를 너무 큰 지연이 생기지 않는 범위에서 적당히 확장 시키고 패킷 폐기 기법을 좀더 보완 한다면 단일 FIFO를 사용하는 FRED 알고리즘에서도 비교적 좋은 성능을 얻을 수 있을 것이다.

제 5 장 결 론

오늘날 인터넷에서는 연속적인 파일 전송과 멀티미디어 스트리밍으로 인해서 다수의 TCP/UDP 플로우가 공존한다. 전송 프로토콜 레벨에서 흐름 제어를 하는 TCP와 그렇지 않은 UDP간의 공정성 향상을 위해서 라우터 버퍼 알고리즘인 기존 RED 알고리즘을 변형한 FRED 알고리즘을 제안 되었다. FRED 알고리즘은 many 옵션 설정을 통해 다수의 flow에 대한 공정성 향상을 위한 기능을 가지고 있다. 그러나 이 역시 동시에 같은 링크에 다수의 TCP/UDP 플로우가 존재 할 경우 불가피하게 TCP 패킷들은 라우터 입력단에서 폐기된다. 이로 인해 전송 프로토콜 레벨에서 혼잡 제어를 지원하는 TCP의 경우 타임 아웃에 걸리게 되고 혼잡 윈도우 크기가 줄어 들어 대역폭 점유율이 급감하고 이에 따라 공정성이 점차 깨지게 된다. 이런 문제를 해결하기 위해 전송 프로토콜 레벨에서 TFRC를 사용하여 흐름제어를 하지 않는 트래픽에 대해 전송측에서 전송률을 TCP와 비슷한 정도로 조절하고 있으나, 모든 전송측에서 사용하지는 않는 방법이다.

본 논문에서는 다수의 TCP/UDP 연결 시 공정성 문제를 시뮬레이션을 통해 살펴보고 보완하기 위해 기존 FRED 알고리즘을 확장한 Many-FRED 알고리즘을 적용하여 실험적으로 적용해 보았다. Many-FRED 알고리즘 제안의 주된 목적인 다수의 서로 다른 트래픽간에 공정성을 보완 하기 위해 FRED에서 제공하는 FIFO 기반의 “two packet buffer” 알고리즘을 변형하여 구현 하였다. 이 방법은 다수의 패킷 폐기를 감지 할 경우 이를 다수의

플로우 연결에 대한 혼잡 상황이라 인지하고 흐름 제어를 하지 않는 트래픽에 대해 현재 평균 큐 크기에 따라 확률적으로 또는 무조건 반입되는 패킷을 폐기 함으로써 높아지는 UDP 트래픽의 대역폭을 줄인다. 따라서 UDP 패킷 폐기를 통한 대역폭을 낮아지는 TCP 트래픽에 대역폭을 할당해 주어서 공정성을 유지하려는 방법이다. 그러나 TCP 혼잡제어 메커니즘에 의해 불가피하게 TCP 트래픽의 전송률은 줄어 들고, 이로 인해 플로우 개수 증가해서 혼잡 상황이 극심해 질수록 TCP 대역폭 점유율이 낮아진다. 비록 Many-FRED을 통해 보다 많은 수의 UDP 패킷을 폐기 한다고 해도 더욱더 많은 수의 플로우 연결이나 혼잡 상황이 발생하면 TCP의 대역폭 점유율은 낮아지고 TCP 플로우별 분산은 증가 할 수 밖에 없다. 4장의 시뮬레이션을 통해 다수의 TCP/UDP 트래픽에 대해 FRED의 many 옵션 사용 시와 Many-FRED 알고리즘 적용 시 결과 분석 자료를 통해 TCP 플로우에서 발생하는 급격한 대역폭 점유율 저하를 약간 완화 할 수 있었고 이에 따라 TCP/UDP 플로우간 대역폭 분산을 낮출 수 있었다. 그러나 혼잡 상황에 의한 Slow Start 진입에 따른 TCP 플로우 별 분산값 증가로 인해 FRED와 Many-FRED 역시 그다지 큰 성능을 발휘 할 수 없었다.

라우터 버퍼 알고리즘에서 대역폭을 보장해 줄 수 있는 방법은 단순히 물리적인 큐를 늘려 주는 방법과 좋은 성능을 보이는 버퍼 관리 알고리즘을 사용하는 방법 있으나, 두 방법 모두 단점이 존재한다. 물리적 메모리 사용으로 값비싼 큐를 확장 하는 것은 제한적이고 큐 크기에 비례하여 지연 시간이 증가 하므로 좋은 방법이 되지를 못한다. 또한 라우터 버퍼 알고리즘

도 한계가 있고 큐에 반입되는 패킷량은 일정하게 한정 되어 있으므로 이 문제를 완벽하게 해결 할 수는 없다. 결론적으로 FRED 알고리즘은 다수의 TCP/UDP 플로우 환경에서는 공정성이 문제가 발생 할 수 있는 소지를 내포하고 있다. 그러나 저렴한 비용과 용이한 구현의 장점이 있으므로, 적절한 크기의 큐 사용과 파라메터 값 설정, 개선된 FRED 알고리즘을 통해 어느 정도 효과적으로 활용 가능 할 것이다.

이 연구의 추후 과제로는 실시간 스트리밍 성능 향상을 위하여 DifferServ를 구성하고 Weighted RED 개념을 사용하여 차등화 서비스 제공으로 문제를 접근할 수 있고, 흐름 제어를 하지 않는 트래픽의 전송률을 제어하는 TFRC, TCP 버전 별 특성을 연구해 전송 프로토콜 레벨에서의 효과적인 혼잡 제어 알고리즘과 여러 종류의 확장된 RED 알고리즘의 호환성을 높이는 연구가 필요하며, 현재 라우터에 많이 사용되는 멀티큐 기반의 라우터 버퍼관리 알고리즘을 연구하여 보완하는 부분이다.

참 고 문 헌

[1] Floyd. S, Jacobson. V, “Random Early Detection for Congestion Avoidance,” IEEE/ACM Transactions on Networking. August 1993.

[2] D. Lin and R. Morris, “Dynamics of Random Early Detection”, Proc. of ACM SIGCOMM, September 1997.

[3] Sally Floyd, Ramakrishna Gummadi, Scott Shenker, “Adaptive RED: An-Algorithm for Increasing the Robustness RED’s Active Queue Manage-ment”, AT&T, 2001.

[4] J. Postel, “Transmission Control Protocol,” RFC 793, September 1981.

[5] J. Postel, “User Datagram Protocol,” RFC 768, August 1980.

[6] W. Richard Stevens, “TCP/IP Illustrated Volume 1, The Protocols,” Addision-Wesley Pub. Co. 1992.

[7] W. Stevens, “TCP Slow Start, Congestion Avoidance, Fast - Retransmit, and Fast Recovery Algorithms,” RFC 2001, January 1997.

[8] Morris, R., “TCP Behavior with Many Flows,” IEEE International Conference on Network Protocols, October 1997.

[9] V. Jacobson. “Modified TCP Congestion Avoidance Algorithm,” Tech-report, 30 Apr 1990.

[10] J. Nagle, “On Packet Switches with Infinite Storage,” IEEE

Transactions on Communications, pp. 435-438, Apr. 1987.

[11] James F.Kurose & Keith W.Ross, “Computer Networking, A Top-Down Approach Featuring the Internet”, Addison Wesley Pub.Co. 2002.

[12] K. Fall, S. Floyd, “Simulation-based Comparisons of Tahoe, Reno, and SACK TCP”, ACM Computer Communication, March, 1996.

[13] Andrew S. Tanenbaum, “Computer Nework Third Edition”, Prentice-Hall Pub. Co. 1998.

[14] B. Braden, “Recommendations on Queue Management and Congestion Avoidance in the Internet,” RFC 2309, April 1998.

[15] Dorgham Sisalem and Adam Wolisz “Towards TCP-friendly Adaptive Multimedia Applications Based on RTP”, in Fourth IEEE Symposium on Computers and Communications (ISCC’ 99), Egypt, July 1999.

[16] D. Sisalem and H. Schulzrinne, “The loss-delay based adjustment algorithm: a TCP-friendly adaptation scheme”, Proceedings of NOSSDAv’98, pp.215-226, July 1998.

[17] Sally Floyd, Mark Handley, Jitendra Padhye and Jorg Widmer “Equation-Based Congestion Control for Unicast Applications”, ACM SIGCOMM, pp.43-56, August 2000.

[18] 배성수, 한종수, “네트워크 시뮬레이터 (NS2 기초와 활용)”, 도서출판 세화, 2005.

[19] 이인규, “인터넷 트래픽위 공정성 향상을 위한 개선된 RED 알고리즘”, 서강대 정보통신대학원, 석사학위논문, 2004, 02.

[20] 구자헌 외, “인터넷 혼잡제어에서 공정성 향상을 위한 새로운 큐 알고리즘”, 한국정보과학회 논문지 1, pp. 0437 ~ 0447, 2003, 06.

[21] 김용범 외, “TCP/UDP 환경에서의 RED 성능 평가”, 한국정보처리학괴 2003년 춘계학술대회, pp, 1229 ~ 1232, 2004, 05.

[22] 낭동호 외, “인터넷 차별화 서비스를 위한 라우터의 공평성 향상 알고리즘”, 한국정보과학회 논문지 1, pp.0358 ~ 0367, 2002, 08.